

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống bãi đỗ xe công cộng với công nghệ
nhận diện biển xe tự động**

ĐỒ ĐÀO PHÚC

phuc.dd210684@sis.hust.edu.vn

Giảng viên hướng dẫn: TS. Đỗ Công Thuần

Chữ kí GVHD

Chương trình đào tạo: Kỹ thuật máy tính (IT2)

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2025

LỜI CẢM ƠN

Lời đầu tiên, cho phép tôi gửi lời cảm ơn đến ban lãnh đạo Trường Công nghệ thông tin và truyền thông đã tạo một môi trường học tập và rèn luyện rất tốt, cung cấp cho tôi những kiến thức, kỹ năng bổ ích để tôi hoàn thành đồ án. Đặc biệt là thầy Đỗ Công Thuần đã tận tâm giúp đỡ tôi trong quá trình nghiên cứu và thực hiện đồ án này.

Tôi cũng gửi lời cảm ơn đến gia đình, người thân đã luôn động viên, ủng hộ và đồng hành cùng tôi trong suốt 4 năm học qua. Bên cạnh đó, tôi còn những người bạn bên cạnh. Xin cảm ơn các bạn đã luôn tin tưởng và hỗ trợ tôi.

Cuối cùng, cảm ơn bản thân tôi vì đã không bỏ cuộc, mặc dù có những lúc mệt mỏi nhưng vẫn luôn cố gắng để vượt qua, hoàn thành chặng đường đại học một cách tốt nhất. Để giờ đây, tôi có thể nói câu tạm biệt với Bách Khoa.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hiện nay, tình trạng thiếu hụt bãi đỗ xe công cộng tại nhiều khu vực ngày càng nghiêm trọng do số lượng phương tiện tăng nhanh. Việc quản lý bãi xe chủ yếu theo phương pháp thủ công gây mất thời gian và khó kiểm soát hiệu quả. Mặc dù đã có một số giải pháp sử dụng camera, cảm biến hay ứng dụng di động, nhưng chi phí cao và độ phổ biến còn hạn chế là những rào cản lớn.

Trong đồ án này, tôi lựa chọn phát triển hệ thống bãi đỗ xe công cộng thông minh dựa trên công nghệ AIoT, tích hợp các thiết bị cảm biến siêu âm, phần cứng nhúng và hệ camera giám sát cùng với thuật toán nhận diện biển số xe tự động bằng YOLO. Giải pháp được thiết kế đơn giản, chi phí thấp nhưng vẫn đáp ứng đầy đủ yêu cầu quản lý và giám sát hiện đại, phù hợp với điều kiện thực tế của các khu vực vừa và nhỏ tại Việt Nam.

Hệ thống sử dụng cảm biến siêu âm và ESP32 để xác định tình trạng các vị trí đỗ xe, kết hợp camera ESP gửi hình ảnh lên server để xử lý nhận diện bằng YOLO, đồng thời RFID hỗ trợ quản lý xe ra vào. Nền tảng web trực quan giúp người dùng theo dõi và cập nhật dữ liệu theo thời gian thực, hỗ trợ vận hành và điều phối bãi xe hiệu quả.

Đồ án đã xây dựng thành công mô hình tích hợp phần cứng và phần mềm cho hệ thống quản lý bãi đỗ xe công cộng thông minh. Kết quả thử nghiệm cho thấy hệ thống vận hành ổn định, dễ mở rộng và có tiềm năng ứng dụng thực tiễn trong các khu dân cư, trường học hoặc trung tâm thương mại.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

ABSTRACT

Currently, the shortage of public parking spaces in many areas is becoming increasingly severe due to the rapid increase in the number of vehicles. Parking lot management is primarily manual, which wastes time and is difficult to control effectively. Although some solutions have been implemented using cameras, sensors, or mobile applications, high costs and limited popularity remain significant barriers.

In this project, I have chosen to develop a smart public parking system based on AIoT technology, integrating ultrasonic sensors, embedded hardware, surveillance cameras, and an automatic vehicle license plate recognition algorithm using YOLO. The solution is designed to be simple, cost-effective, yet still meets all the requirements for modern management and surveillance, making it suitable for the practical conditions of medium and small areas in Vietnam.

The system uses ultrasonic sensors and ESP32 to determine the status of parking spots, combined with ESP cameras to send images to a server for YOLO-based recognition, while RFID helps manage vehicle entry and exit. The intuitive web platform allows users to monitor and update data in real-time, supporting efficient parking lot operation and coordination.

The project successfully developed a model integrating both hardware and software for the smart public parking management system. Experimental results show that the system operates stably, is easily scalable, and has practical application potential in residential areas, schools, or shopping centers.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Quy trình nghiệp vụ: Gửi xe tự động.....	7
2.3 Đặc tả chức năng	8
2.3.1 Đặc tả use case: Đăng ký và cập nhật thông tin	9
2.3.2 Đặc tả use case: Nạp tiền vào tài khoản	10
2.3.3 Đặc tả use case: Quản lý ra/vào bãi xe.....	11
2.3.4 Đặc tả use case: Thống kê doanh thu.....	12
2.3.5 Đặc tả use case: Tra cứu lịch sử đỗ xe	13
2.3.6 Đặc tả use case: Quản lý người dùng.....	14
2.4 Yêu cầu phi chức năng	14
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	16
3.1 YOLOv8 – Nhận diện biển số xe.....	16
3.2 Django – Nền tảng phát triển Web.....	17
3.3 SQLite3 – Hệ quản trị cơ sở dữ liệu.....	17
3.4 Cảm biến hồng ngoại LM393 – Phát hiện vật cản	18
3.5 Module WiFi + BLE ESP32 NodeMCU LuaNode32.....	19

3.6 Module RFID RC522 13.56MHz	20
3.7 Servo Tower Pro SG90 – Mô tơ điều khiển đóng/mở cổng	21
3.8 ESP32-CAM – Camera xử lý hình ảnh và truyền dữ liệu	22
3.9 Màn hình e-Ink 4.2 inch – Hiển thị trạng thái bãi đỗ và mã QR thanh toán .	23
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	25
4.1 Thiết kế kiến trúc.....	25
4.1.1 Lựa chọn kiến trúc phần mềm	25
4.1.2 Thiết kế tổng quan.....	27
4.1.3 Thiết kế chi tiết gói	28
4.2 Thiết kế chi tiết.....	36
4.2.1 Thiết kế giao diện	36
4.2.2 Thiết kế lớp	40
4.2.3 Thiết kế cơ sở dữ liệu	41
4.2.4 Thiết kế phần cứng.....	43
4.3 Xây dựng ứng dụng.....	44
4.3.1 Thư viện và công cụ sử dụng.....	44
4.3.2 Kết quả đạt được	46
4.3.3 Ý nghĩa và vai trò của các thành phần	46
4.3.4 Kết quả đạt được	47
4.3.5 Minh họa các chức năng chính	47
4.4 Kiểm thử.....	49
4.4.1 Nhận diện biến số và mở cổng.....	49
4.4.2 Thanh toán tự động qua mã QR	50
4.4.3 Tổng kết kiểm thử	52
4.5 Triển khai	52

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	53
5.1 Phát hiện và nhận diện biến số xe	53
5.1.1 Giới thiệu về bài toán/ vấn đề	53
5.1.2 Giải pháp	53
5.1.3 Kết quả đạt được	56
5.2 Kết nối phần cứng và phần mềm	57
5.2.1 Giới thiệu vấn đề.....	57
5.2.2 Giải pháp triển khai.....	57
5.2.3 Kết quả đạt được	60
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
6.1 Kết luận.....	61
6.2 Hướng phát triển.....	61

DANH MỤC HÌNH VẼ

Hình 2.1	Hình ảnh mô phỏng hệ thống bãi đỗ xe thông minh	6
Hình 2.2	Biểu đồ use case tổng quát	7
Hình 3.1	Hình ảnh YOLOv8	17
Hình 3.2	Hình ảnh django kết hợp python	17
Hình 3.3	Hình ảnh SQLite3	18
Hình 3.4	Hình ảnh cảm biến hồng ngoại LM393	19
Hình 3.5	Hình ảnh ESP32 NodeMCU LuaNode32	20
Hình 3.6	Hình ảnh Module RFID RC522	21
Hình 3.7	Hình ảnh servo SG90	22
Hình 3.8	Hình ảnh ESP32 cam	23
Hình 3.9	Hình ảnh màn hình e-link 4.2 inch	24
Hình 4.1	Sơ đồ tổng quan thiết kế của hệ thống	27
Hình 4.2	Mô hình thiết kế tổng quan phần mềm	27
Hình 4.3	Thiết kế chi tiết gói core	28
Hình 4.4	Thiết kế chi tiết gói app	30
Hình 4.5	Thiết kế chi tiết gói access	31
Hình 4.6	Thiết kế chi tiết gói authentication	32
Hình 4.7	Thiết kế chi tiết gói customers	33
Hình 4.8	Thiết kế chi tiết gói mqttcall	34
Hình 4.9	Thiết kế chi tiết gói tcp_server	35
Hình 4.10	Thiết kế chi tiết gói camera	36
Hình 4.11	Màn hình minh họa trang quản trị	37
Hình 4.12	Màn hình minh họa lịch sử ra vào	38
Hình 4.13	Màn hình minh họa quản lý người dùng	38
Hình 4.14	Màn hình minh họa thống kê doanh thu	39
Hình 4.15	Màn hình minh họa hồ sơ người dùng	39
Hình 4.16	Biểu đồ thực thể liên kết	42
Hình 4.17	Màn hình cập nhật thông tin	47
Hình 4.18	Màn hình quản lý ra vào	48
Hình 4.19	Màn hình thống kê lịch sử ra vào	48
Hình 4.20	Hình ảnh phần cứng của hệ thống	49
Hình 5.1	Hình ảnh kiến trúc YOLOv8	54
Hình 5.2	Huấn luyện mô hình nhận diện biển xe	55

Hình 5.3	Huấn luyện mô hình nhận diện ký tự trong biển số xe	56
Hình 5.4	Đánh giá độ chính xác	56
Hình 5.5	Hàm gửi 1 khung hình dạng JPEG qua HTTP	57
Hình 5.6	Hàm xử lý nhận diện biển số xe	58
Hình 5.7	ESP nhận lệnh đóng mở cổng	59
Hình 5.8	Server gửi thông điệp đóng mở cổng	60

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case: Đăng ký và cập nhật thông tin	9
Bảng 2.2	Đặc tả use case Nạp tiền	10
Bảng 2.3	Đặc tả use case Quản lý ra/vào bãi xe	11
Bảng 2.4	Đặc tả use case Thống kê doanh thu	12
Bảng 2.5	Đặc tả use case Tra cứu lịch sử đỗ xe	13
Bảng 2.6	Đặc tả use case Quản lý người dùng	14
Bảng 4.1	Thuộc tính lớp Profile	40
Bảng 4.2	Thuộc tính lớp TopUpHistory	40
Bảng 4.3	Thuộc tính lớp AccessHistory	41
Bảng 4.4	Thuộc tính lớp Camera	41
Bảng 4.5	Sơ đồ kết nối phần cứng với ESP32	43
Bảng 4.6	Kết nối giữa ESP32 và màn hình E-Paper 4.2 inch	43
Bảng 4.7	Danh sách thư viện và công cụ sử dụng	45
Bảng 4.8	Thông tin thông kê ứng dụng	46

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
ASGI	Giao diện máy chủ web không đồng bộ (Asynchronous Server Gateway Interface)
BLE	Bluetooth Low Energy - Công nghệ Bluetooth tiết kiệm năng lượng
CNN	Mạng nơ-ron tích chập (Convolutional Neural Network)
CPU	Bộ xử lý trung tâm (Central Processing Unit)
CSDL	Cơ sở dữ liệu (Database)
CSS	Cascading Style Sheets (CSS) - Các bảng kiểu định dạng cascading
GPU	Bộ xử lý đồ họa (Graphics Processing Unit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IDE	Môi trường phát triển tích hợp (Integrated Development Environment)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương 1 có độ dài từ 3 đến 6 trang với các nội dung sau đây

1.1 Đặt vấn đề

Trong bối cảnh đô thị hóa ngày càng diễn ra mạnh mẽ tại các thành phố lớn cũng như khu vực đang phát triển, nhu cầu sử dụng phương tiện cá nhân không ngừng tăng lên. Tỷ lệ sở hữu ô tô và xe máy tại Việt Nam có xu hướng gia tăng nhanh chóng, dẫn đến áp lực lớn đối với cơ sở hạ tầng giao thông, đặc biệt là không gian dành cho bãi đỗ xe công cộng. Nhiều khu vực dân cư, trung tâm thương mại, cơ quan hành chính hoặc trường học thường xuyên lâm vào tình trạng thiếu hụt vị trí đỗ xe, gây cản trở giao thông, tăng nguy cơ ùn tắc và làm giảm chất lượng sống đô thị.

Thực tế hiện nay, phần lớn các bãi đỗ xe vẫn đang được quản lý bằng phương pháp thủ công, phụ thuộc vào nhân sự trực tiếp ghi nhận và điều phối, khiến quy trình vận hành kém hiệu quả, dễ phát sinh sai sót và khó tích hợp với các nền tảng số hiện đại. Việc tìm kiếm chỗ đỗ xe trống cũng trở thành gánh nặng cho người dân, đặc biệt trong các khung giờ cao điểm hoặc tại những khu vực có mật độ phương tiện cao. Điều này không chỉ làm tăng thời gian di chuyển mà còn góp phần phát sinh thêm lượng khí thải từ các phương tiện đang tìm chỗ đỗ, ảnh hưởng tiêu cực đến môi trường và sức khỏe cộng đồng.

Nếu vấn đề này được giải quyết một cách hiệu quả, không gian đô thị sẽ được sử dụng hợp lý hơn, lưu lượng giao thông được điều tiết trật tự và người dân sẽ tiết kiệm được thời gian. Đồng thời, đây cũng là tiền đề để xây dựng các mô hình đô thị thông minh, thân thiện với môi trường, lấy công nghệ làm nền tảng hỗ trợ cho quản lý hạ tầng. Hệ thống bãi đỗ xe thông minh là một trong những giải pháp thiết yếu trong định hướng phát triển đô thị bền vững. Các mô hình như vậy có thể được triển khai linh hoạt tại nhiều địa điểm khác nhau như khu dân cư, khu công nghiệp, bệnh viện, sân bay hay nhà ga, với khả năng mở rộng và tích hợp vào các hệ thống quản lý tổng thể trong tương lai.

1.2 Mục tiêu và phạm vi đề tài

Dựa trên các nghiên cứu và quá trình tìm hiểu, nhiều giải pháp đã được triển khai để giải quyết vấn đề thiếu bãi đỗ xe, chẳng hạn như việc sử dụng camera giám sát và RFID. Một số trường học đã tích hợp FRID vào thẻ sinh viên, dùng để ra vào bãi gửi xe. Tuy nhiên, các giải pháp này vẫn còn nhiều hạn chế, đặc biệt là phải trực tiếp đến phòng quản lý để nạp tiền vào thẻ. Và khi sinh viên không nạp tiền

thì sẽ phải đưa tiền mặt, mất nhiều thời gian, gây ùn tắc.

Trên cơ sở phân tích các giải pháp hiện có, mục tiêu của đề tài này là phát triển một hệ thống quản lý bãi đỗ xe công cộng thông minh sử dụng công nghệ AIoT. Đề tài này tập trung vào việc thiết kế một hệ thống đơn giản, chi phí thấp nhưng vẫn đáp ứng đầy đủ các yêu cầu cơ bản của một hệ thống bãi đỗ xe thông minh. Hệ thống sẽ sử dụng các cảm biến siêu âm để xác định tình trạng các vị trí đỗ, vi điều khiển ESP32 để xử lý dữ liệu, RFID để hỗ trợ nhận diện phương tiện và nền tảng web để người dùng theo dõi, nạp tiền và thanh toán tự động. Hệ thống tập giải quyết các bất cập hiện tại, hướng đến khả năng mở rộng linh hoạt, áp dụng thực tế vào các trường học, cơ quan và các bãi đỗ xe công cộng.

1.3 Định hướng giải pháp

Để giải quyết vấn đề thiếu hụt bãi đỗ xe công cộng, đề tài này sẽ được tiếp cận theo hướng kết hợp các công nghệ phần mềm và phần cứng như các thiết bị cảm biến siêu âm, vi điều khiển ESP32, camera ESP và RFID. Phương pháp chính được áp dụng là nhận diện xe bằng camera kết hợp thuật toán YOLO, cảm biến siêu âm để phát hiện sự hiện diện của xe và hệ thống RFID để quản lý xe ra vào. Dữ liệu từ các thiết bị sẽ được đồng bộ và hiển thị trên nền tảng web, cho phép người dùng theo dõi lịch sử nạp tiền, lịch sử đỗ xe, người quản lý có thể theo dõi được doanh thu và quản lý ra vào.

Đóng góp chính của đồ án là phát triển một mô hình bãi đỗ xe công cộng thông minh, dễ triển khai, chi phí thấp và khả năng mở rộng cao, giảm thiểu sự can thiệp từ con người, mang lại hiệu quả quản lý bãi đỗ xe tốt hơn và có thể áp dụng thực tế trong các khu vực đô thị.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

i. Chương 2 trình bày về hiện trạng quản lý bãi đỗ xe công cộng hiện nay. Nội dung chương tập trung phân tích các yêu cầu thực tiễn đặt ra đối với một hệ thống bãi đỗ xe thông minh, từ đó xác định các chức năng cần có và tiêu chí kỹ thuật cần đáp ứng trong quá trình phát triển hệ thống. Việc đánh giá hiện trạng và đề xuất yêu cầu đóng vai trò làm cơ sở định hướng cho toàn bộ quá trình thiết kế và triển khai đồ án.

ii. Chương 3 cung cấp cơ sở lý thuyết và trình bày tổng quan các công nghệ cốt lõi phục vụ cho việc xây dựng hệ thống. Trong chương này, đồ án sẽ giới thiệu chi tiết về các thiết bị nhúng được lựa chọn như vi điều khiển ESP32, cảm biến siêu âm, camera ESP, module RFID, cũng như mô hình và thuật toán nhận diện

đối tượng bằng YOLO. Ngoài ra, các nguyên lý hoạt động, ưu nhược điểm và tính phù hợp của từng công nghệ trong bối cảnh hệ thống bãi đỗ xe thông minh cũng sẽ được phân tích cụ thể.

ii. Chương 4 tập trung vào quá trình thiết kế và triển khai hệ thống. Nội dung bao gồm việc xây dựng sơ đồ hệ thống tổng thể, thiết kế phần cứng, lập trình cho các thiết bị nhúng, triển khai nền tảng phần mềm quản lý, và kết nối các thành phần thành một hệ thống hoàn chỉnh. Chương này cũng trình bày quy trình kiểm thử hệ thống, đánh giá kết quả vận hành, độ chính xác của cảm biến và mô hình nhận diện, cũng như hiệu năng phản hồi của giao diện người dùng.

iv. Chương 5 là chương cuối cùng, tổng kết toàn bộ quá trình thực hiện đồ án. Nội dung chương bao gồm đánh giá hiệu quả giải pháp đề xuất, chỉ ra những thách thức gặp phải trong quá trình triển khai thực tế và cách thức khắc phục. Đồng thời, chương này cũng đưa ra một số đề xuất nhằm cải tiến hệ thống, mở rộng phạm vi ứng dụng và phát triển thêm các chức năng trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 2 trình bày khảo sát hiện trạng về nhu cầu và các bất cập trong quản lý bãi đỗ xe công cộng, từ góc nhìn của người dùng, người quản lý và các hệ thống hiện có. Dựa trên khảo sát này, chương đưa ra tổng quan các chức năng chính của hệ thống bãi đỗ xe thông minh, đồng thời phân tích và mô tả chi tiết các use case quan trọng. Các quy trình nghiệp vụ then chốt, đặc biệt là quy trình gửi xe tự động, được minh họa bằng biểu đồ hoạt động để làm rõ cách thức phối hợp các thành phần trong hệ thống. Cuối cùng, chương trình bày các yêu cầu phi chức năng nhằm đảm bảo hệ thống vận hành ổn định, hiệu quả và thân thiện với người dùng.

2.1 Khảo sát hiện trạng

Việc khảo sát hiện trạng là bước quan trọng nhằm xác định nhu cầu thực tế, những vấn đề tồn tại và các yêu cầu chức năng cần thiết cho hệ thống bãi đỗ xe thông minh. Khảo sát trong đề tài này được tiến hành từ ba nguồn chính: người dùng (người gửi xe), người quản lý bãi xe, và các hệ thống, ứng dụng bãi đỗ xe hiện có trên thị trường. Kết quả khảo sát là cơ sở để xây dựng hệ thống phù hợp với điều kiện triển khai thực tế tại các khu vực có quy mô vừa và nhỏ.

Khảo sát người dùng

Đối tượng khảo sát là người sử dụng phương tiện cá nhân có nhu cầu gửi xe tại các khu vực công cộng như trường học, khu dân cư, hoặc cơ quan hành chính. Theo kết quả khảo sát, đất nước ta đã vượt mức 4,5 triệu xe ô tô, diện tích đất dành cho các bãi đỗ xe chỉ chiếm 1% quỹ đất quy mô từ đó người dùng gặp phải một số bất tiện phổ biến như: ùn tắc mất thời gian chờ mở cổng ngoài ra còn không có phương thức thanh toán linh hoạt, không theo dõi được lịch sử gửi xe, hoặc dễ xảy ra nhầm lẫn nếu gửi xe nhiều lần trong ngày.

Người dùng mong muốn có một hệ thống tự động nhận diện biển số để ra vào nhanh chóng, không cần phải quẹt thẻ hoặc ghi phiếu thủ công. Đặc biệt, nhu cầu về tích hợp thanh toán, hỗ trợ nạp tiền và trừ phí gửi xe tự động cũng được nhiều người đánh giá cao vì giúp tiết kiệm thời gian và minh bạch chi phí. Ngoài ra, việc xem lại lịch sử ra vào, chi tiết thời gian và số tiền đã trừ cũng được xem là cần thiết để kiểm soát chi tiêu và tránh tranh chấp.

Khảo sát người quản lý bãi xe

Người quản lý hiện nay chủ yếu điều hành bãi xe bằng hình thức ghi sổ, phát phiếu hoặc quẹt thẻ RFID rời. Việc ghi nhận thủ công khiến dữ liệu thiếu chính xác, dễ thất lạc hoặc bị chỉnh sửa. Trong trường hợp cao điểm, khó kiểm soát được

số lượng xe ra vào cũng như thu phí đúng với thời gian thực tế. Ngoài ra, việc thu tiền mặt trực tiếp vừa tốn công sức, vừa không đảm bảo tính minh bạch.

Qua trao đổi với người quản lý, hệ thống lý tưởng nên bao gồm khả năng: tự động nhận diện biển số để xe đã đăng ký có thể ra/vào nhanh chóng; sử dụng RFID như một giải pháp dự phòng cho người chưa đăng ký; thống kê chính xác số lượt gửi xe trong ngày; tự động trừ tiền từ tài khoản; và có thể theo dõi lịch sử gửi xe của từng phương tiện. Ngoài ra, giao diện quản lý phải đơn giản, dễ sử dụng và hoạt động ổn định trong điều kiện mạng nội bộ hoặc kết nối Wi-Fi thông thường.

Khảo sát các hệ thống, ứng dụng hiện có

Hiện nay, một số hệ thống bãi đỗ xe thông minh đã được triển khai, chủ yếu tại các trung tâm thương mại hoặc tòa nhà văn phòng. Một số giải pháp nổi bật bao gồm:

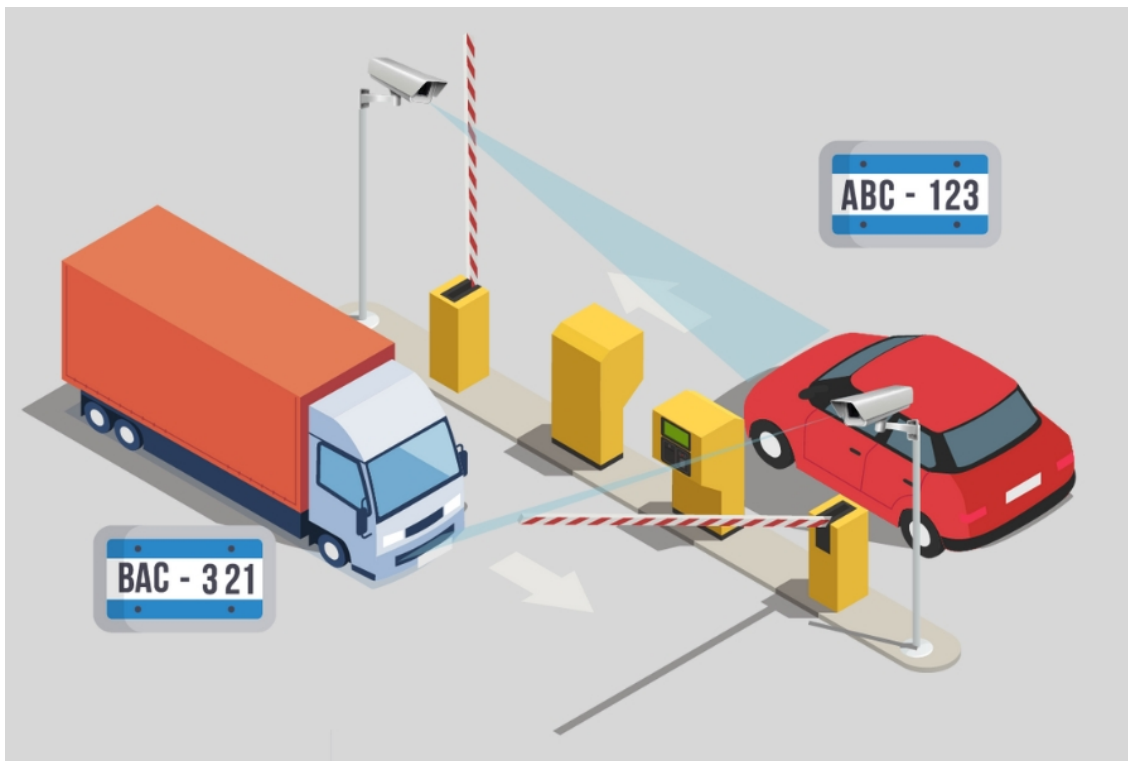
Hệ thống camera chụp lại biển số xe và nhân viên ấn nút mở cổng.

Hệ thống sử dụng thẻ từ hoặc RFID để kiểm soát ra vào và ghi nhận thời gian gửi xe.

Một số hệ thống có tích hợp phần mềm quản lý thu phí theo lượt hoặc theo giờ, nhưng vẫn yêu cầu thanh toán thủ công hoặc dùng máy POS.

Tuy nhiên, đa số các hệ thống trên không tích hợp chức năng nạp tiền trực tuyến, không cung cấp giao diện cho người dùng theo dõi lịch sử ra/vào, và chưa hỗ trợ thanh toán tự động theo mô hình tài khoản định danh.

Từ những khảo sát và đánh giá nêu trên, hệ thống cần hướng đến mô hình quản lý hiện đại: cho phép đăng ký tài khoản theo biển số xe, tự động mở cổng khi phát hiện xe đã đăng ký, sử dụng RFID như một giải pháp thay thế khi chưa đăng ký, hỗ trợ người dùng nạp tiền vào ví điện tử, trừ phí tự động, và cung cấp giao diện web theo dõi lịch sử giao dịch và ra vào.



Hình 2.1: Hình ảnh mô phỏng hệ thống bãi đỗ xe thông minh

2.2 Tổng quan chức năng

Phần 2.2 này trình bày tổng quan các chức năng chính của hệ thống bãi đỗ xe thông minh. Hệ thống được thiết kế nhằm phục vụ hai nhóm người dùng chính: người dùng (người gửi xe) và người quản trị (người quản lý bãi xe). Hệ thống hoạt động dựa trên nguyên tắc nhận diện biển số tự động kết hợp RFID, hỗ trợ thanh toán qua tài khoản nội bộ và ghi lại lịch sử xe ra/vào.

Các chức năng chính của hệ thống bao gồm:

- Đăng ký tài khoản người dùng và liên kết với biển số xe.
- Ghi nhận xe ra/vào bằng nhận diện biển số hoặc RFID.
- Tự động kiểm tra số dư tài khoản và thanh toán.
- Nạp tiền vào tài khoản gửi xe.
- Quản lý, thống kê doanh thu và tra cứu lịch sử ra/vào.

2.2.1 Biểu đồ use case tổng quát



Hình 2.2: Biểu đồ use case tổng quát

Trong biểu đồ use case tổng quát, hệ thống có hai tác nhân chính:

Người dùng: là cá nhân sử dụng phương tiện và đã đăng ký tài khoản. Người dùng có thể truy cập để đăng ký tài khoản, nạp tiền, theo dõi lịch sử ra/vào, và hệ thống tự động ghi nhận khi xe ra/vào bằng biển số.

Người quản lý: là người điều hành hệ thống tại bãi xe, có quyền xem thống kê, theo dõi hoạt động, kiểm tra lịch sử ra/vào.

Các use case chính gồm:

- Chỉnh sửa thông tin
- Nạp tiền
- Quản lý ra/vào
- Thống kê doanh thu
- Tra cứu lịch sử đỗ xe
- Quản lý người dùng

2.2.2 Quy trình nghiệp vụ: Gửi xe tự động

Trong hệ thống bãi đỗ xe thông minh được xây dựng, quy trình gửi xe tự động là một trong những nghiệp vụ quan trọng, kết hợp nhiều chức năng và thành phần khác nhau trong hệ thống. Quy trình này đảm bảo rằng người dùng có thể vào và ra khỏi bãi đỗ xe một cách nhanh chóng, chính xác, an toàn và hoàn toàn tự động nếu đã đăng ký tài khoản, hoặc sử dụng RFID nếu chưa đăng ký.

Quy trình bao gồm các bước chính như sau:

- i. Người dùng điều khiển phương tiện đến cổng bãi xe.
- ii. Hệ thống kích hoạt camera chụp ảnh và nhận diện biển số xe.
- iii. Nếu biển số đã đăng ký trong hệ thống:
 - Hệ thống xác thực thông tin tài khoản.
 - Nếu tài khoản hợp lệ, hệ thống tự động mở cổng và ghi nhận thời điểm vào.
- iv. Nếu biển số chưa được đăng ký:
 - Hệ thống yêu cầu người dùng quét thẻ RFID.
 - Nếu thẻ hợp lệ, hệ thống mở cổng và ghi nhận thời điểm vào/ra.
- v. Khi xe rời khỏi bãi:
 - Hệ thống xác định thời gian đỗ và tính phí tương ứng.
 - Trừ tiền trong tài khoản người dùng (nếu có).
- vi. Nếu không đủ tiền hoặc dùng RFID
 - Hệ thống hiển thị mã QR để người dùng thanh toán
 - Nếu thanh toán thành công thì xe được đi
- vii. Ghi nhận toàn bộ thông tin vào cơ sở dữ liệu và cập nhật lịch sử gửi xe.

Quản lý có thể tra cứu nhật ký xe ra/vào và kiểm tra doanh thu theo từng phiên làm việc hoặc theo thời gian thực.

Quy trình này đảm bảo tính liên tục và tự động hóa cao, giúp giảm thiểu nhân lực vận hành, nâng cao trải nghiệm người dùng và tăng tính minh bạch trong việc tính phí đỗ xe.

2.3 Đặc tả chức năng

Phần này trình bày đặc tả chi tiết các chức năng chính trong hệ thống, được lựa chọn dựa trên mức độ quan trọng và tần suất sử dụng. Mỗi use case thể hiện rõ cách hệ thống tương tác với người dùng để thực hiện một nghiệp vụ cụ thể..

2.3.1 Đặc tả use case: Đăng ký và cập nhật thông tin

Tên use case	Đăng ký tài khoản	
Tác nhân	Người dùng	
Mô tả	Người dùng tạo tài khoản mới bằng cách nhập username, email, mật khẩu, sau đó cập nhật thông tin cá nhân gồm họ tên, số điện thoại và biển số xe để sử dụng hệ thống quản lý bãi đỗ xe thông minh.	
Tiền điều kiện	Người dùng chưa có đăng ký tài khoản.	
Hậu điều kiện	Tài khoản người dùng được tạo thành công, thông tin cá nhân được cập nhật đầy đủ và có thể sử dụng để đăng nhập hệ thống.	
Luồng cơ bản	Tác nhân 1. Truy cập giao diện đăng ký 3. Nhập username, email, mật khẩu 6. Nhập họ tên, số điện thoại, biển số xe	Hệ thống 2. Hiện thị mẫu đăng ký 4. Kiểm tra hợp lệ và xác minh username/email chưa tồn tại 5. Thông báo đăng ký thành công 7. Kiểm tra hợp lệ và lưu thông tin 8. Thông báo cập nhật thành công
Luồng thay đổi	Nếu thông tin ban đầu không hợp lệ hoặc username/email đã tồn tại, hệ thống hiển thị thông báo lỗi và yêu cầu nhập lại.	
Ngoại lệ	Người dùng huỷ đăng ký hoặc thoát giữa chừng trong quá trình nhập thông tin.	

Bảng 2.1: Đặc tả use case: Đăng ký và cập nhật thông tin

2.3.2 Đặc tả use case: Nạp tiền vào tài khoản

Tên use case	Nạp tiền										
Tác nhân	Người dùng										
Mô tả	Người dùng thực hiện nạp tiền vào tài khoản để sử dụng cho việc thanh toán tự động khi gửi xe										
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống										
Hậu điều kiện	Số dư trong tài khoản người dùng được cập nhật chính xác sau khi hoàn tất thanh toán										
Luồng cơ bản	<table border="1"> <thead> <tr> <th>Tác nhân</th><th>Hệ thống</th></tr> </thead> <tbody> <tr> <td>1. Người dùng chọn chức năng “Nạp tiền”</td><td>2. Hiển thị mẫu nhập số tiền</td></tr> <tr> <td>3. Người dùng nhập số tiền</td><td>4. Tạo mã QR hoặc hướng dẫn thanh toán</td></tr> <tr> <td>5. Người dùng quét mã và hoàn tất thanh toán</td><td>6. Nhận thông báo (web-hook) từ nhà cung cấp thanh toán</td></tr> <tr> <td></td><td>7. Hiển thị thông báo nạp tiền thành công và cập nhật số dư</td></tr> </tbody> </table>	Tác nhân	Hệ thống	1. Người dùng chọn chức năng “Nạp tiền”	2. Hiển thị mẫu nhập số tiền	3. Người dùng nhập số tiền	4. Tạo mã QR hoặc hướng dẫn thanh toán	5. Người dùng quét mã và hoàn tất thanh toán	6. Nhận thông báo (web-hook) từ nhà cung cấp thanh toán		7. Hiển thị thông báo nạp tiền thành công và cập nhật số dư
Tác nhân	Hệ thống										
1. Người dùng chọn chức năng “Nạp tiền”	2. Hiển thị mẫu nhập số tiền										
3. Người dùng nhập số tiền	4. Tạo mã QR hoặc hướng dẫn thanh toán										
5. Người dùng quét mã và hoàn tất thanh toán	6. Nhận thông báo (web-hook) từ nhà cung cấp thanh toán										
	7. Hiển thị thông báo nạp tiền thành công và cập nhật số dư										
Luồng thay đổi	Nếu giao dịch thất bại, hệ thống không cập nhật số dư và hiển thị thông báo lỗi; nếu người dùng hủy giao dịch, phiên nạp tiền bị hủy bỏ										
Ngoại lệ	Nếu có lỗi mạng hoặc mất kết nối trong quá trình thanh toán, giao dịch được thông báo thất bại và không cập nhật số dư										

Bảng 2.2: Đặc tả use case Nạp tiền

2.3.3 Đặc tả use case: Quản lý ra/vào bãi xe

Tên use case	Quản lý ra/vào bãi xe	
Tác nhân	Người quản lý, người dùng	
Mô tả	Hệ thống kiểm soát việc xe ra vào bãi đỗ dựa trên biển số đã đăng ký hoặc thẻ RFID, tự động tính phí và quản lý lịch sử gửi xe	
Tiền điều kiện	Hệ thống hoạt động bình thường, camera và cảm biến sẵn sàng; người dùng đã đăng ký tài khoản với biển số hoặc có thẻ RFID	
Hậu điều kiện	Xe được ghi nhận ra/vào, phí gửi xe được tính và trừ tự động hoặc yêu cầu thanh toán qua QR khi cần; lịch sử gửi xe được lưu trữ	
Luồng cơ bản	Tác nhân 1. Người dùng đến cổng với xe đã đăng ký biển số 4. Người dùng lấy xe đến cổng ra 6. Người dùng thanh toán thành công	Hệ thống 2. Hệ thống nhận diện và xác thực biển số 3. Mở cổng cho xe vào, ghi nhận thời gian vào 5. Tự động trừ phí nếu tài khoản đủ, nếu không sẽ hiện mã thanh toán 7. Mở cổng cho xe ra và lưu lại lịch sử
Luồng thay đổi	Tác nhân 1. Người dùng đến cổng với xe chưa đăng ký biển số 3. Người dùng quét thẻ RFID để vào bãi 3. Khi ra bãi, người dùng quét thẻ RFID lần nữa 6. Người dùng thanh toán thành công	Hệ thống 2. Hệ thống yêu cầu quét thẻ RFID 4. Ghi nhận thời điểm vào bãi 4. Tính phí gửi xe dựa trên thời gian sử dụng 5. Hiển thị mã QR để thanh toán phí gửi xe 7. Mở cổng cho xe ra và lưu lại lịch sử
Ngoại lệ	Lỗi nhận diện biển số, lỗi thẻ RFID, lỗi kết nối mạng hoặc lỗi hệ thống cảm biến	

Bảng 2.3: Đặc tả use case Quản lý ra/vào bãi xe

2.3.4 Đặc tả use case: Thống kê doanh thu

Tên use case	Thống kê doanh thu										
Tác nhân	Người quản lý										
Mô tả	Người quản lý theo dõi và xem báo cáo thống kê doanh thu từ các lượt gửi xe trong hệ thống trong khoảng thời gian được chọn										
Tiền điều kiện	Người quản lý đã đăng nhập hệ thống và có quyền truy cập chức năng thống kê										
Hậu điều kiện	Báo cáo doanh thu được hiển thị, có thể sử dụng cho việc phân tích, quản lý										
Luồng cơ bản	<table border="0"> <thead> <tr> <th>Tác nhân</th><th>Hệ thống</th></tr> </thead> <tbody> <tr> <td>1. Người quản lý chọn chức năng thống kê doanh thu</td><td>2. Hiển thị giao diện nhập khoảng thời gian</td></tr> <tr> <td>3. Người quản lý nhập khoảng thời gian hoặc loại khách hàng cần thống kê</td><td>4. Truy xuất dữ liệu lượt gửi xe trong khoảng thời gian</td></tr> <tr> <td></td><td>5. Tính tổng doanh thu từ các lượt gửi xe</td></tr> <tr> <td></td><td>6. Hiển thị báo cáo doanh thu</td></tr> </tbody> </table>	Tác nhân	Hệ thống	1. Người quản lý chọn chức năng thống kê doanh thu	2. Hiển thị giao diện nhập khoảng thời gian	3. Người quản lý nhập khoảng thời gian hoặc loại khách hàng cần thống kê	4. Truy xuất dữ liệu lượt gửi xe trong khoảng thời gian		5. Tính tổng doanh thu từ các lượt gửi xe		6. Hiển thị báo cáo doanh thu
Tác nhân	Hệ thống										
1. Người quản lý chọn chức năng thống kê doanh thu	2. Hiển thị giao diện nhập khoảng thời gian										
3. Người quản lý nhập khoảng thời gian hoặc loại khách hàng cần thống kê	4. Truy xuất dữ liệu lượt gửi xe trong khoảng thời gian										
	5. Tính tổng doanh thu từ các lượt gửi xe										
	6. Hiển thị báo cáo doanh thu										
Luồng thay đổi	Nếu không có dữ liệu trong khoảng thời gian chọn, hiển thị thông báo “Không có giao dịch”										
Ngoại lệ	Lỗi truy xuất dữ liệu hoặc lỗi hệ thống khi tính toán, hiển thị thông báo lỗi										

Bảng 2.4: Đặc tả use case Thống kê doanh thu

2.3.5 Đặc tả use case: Tra cứu lịch sử đỗ xe

Tên use case	Tra cứu lịch sử đỗ xe	
Tác nhân	Người dùng, Người quản lý	
Mô tả	Người dùng hoặc người quản lý truy cập và xem lại lịch sử các lượt gửi xe, bao gồm thời gian vào/ra, phí đã thanh toán	
Tiền điều kiện	Người dùng hoặc người quản lý đã đăng nhập hệ thống	
Hậu điều kiện	Hiển thị đầy đủ lịch sử gửi xe theo yêu cầu, có thể lọc và tìm kiếm	
Luồng cơ bản	Tác nhân 1. Người dùng hoặc quản lý chọn chức năng xem lịch sử gửi xe 3. Người dùng nhập tiêu chí lọc (ngày, biển số, mã thẻ RFID)	Hệ thống 2. Hiển thị giao diện lịch sử gửi xe 4. Truy xuất dữ liệu lịch sử theo tiêu chí 5. Hiển thị danh sách lịch sử gửi xe
Luồng thay đổi	Nếu không có dữ liệu phù hợp tiêu chí lọc, hiển thị thông báo “Không có dữ liệu”	
Ngoại lệ	Lỗi truy xuất dữ liệu hoặc lỗi hệ thống, hiển thị thông báo lỗi	

Bảng 2.5: Đặc tả use case Tra cứu lịch sử đỗ xe

2.3.6 Đặc tả use case: Quản lý người dùng

Tên use case	Quản lý người dùng										
Tác nhân	Người quản lý										
Mô tả	Người quản lý xem, chỉnh sửa thông tin người dùng và có thể cấp lại mật khẩu khi cần thiết										
Tiền điều kiện	Người quản lý đã đăng nhập hệ thống với quyền quản trị										
Hậu điều kiện	Thông tin người dùng được cập nhật chính xác; mật khẩu được cấp lại thành công nếu thực hiện										
Luồng cơ bản	<table border="0"> <thead> <tr> <th>Tác nhân</th><th>Hệ thống</th></tr> </thead> <tbody> <tr> <td>1. Người quản lý truy cập danh sách người dùng</td><td>2. Hiển thị danh sách người dùng</td></tr> <tr> <td>3. Người quản lý chọn người dùng để xem/chỉnh sửa thông tin</td><td>4. Hiển thị chi tiết thông tin người dùng</td></tr> <tr> <td>5. Người quản lý cập nhật thông tin (nếu cần)</td><td>6. Lưu thông tin cập nhật</td></tr> <tr> <td>7. Người quản lý cấp lại mật khẩu mới cho người dùng</td><td>8. Thông báo thành công và cập nhật mật khẩu mới</td></tr> </tbody> </table>	Tác nhân	Hệ thống	1. Người quản lý truy cập danh sách người dùng	2. Hiển thị danh sách người dùng	3. Người quản lý chọn người dùng để xem/chỉnh sửa thông tin	4. Hiển thị chi tiết thông tin người dùng	5. Người quản lý cập nhật thông tin (nếu cần)	6. Lưu thông tin cập nhật	7. Người quản lý cấp lại mật khẩu mới cho người dùng	8. Thông báo thành công và cập nhật mật khẩu mới
Tác nhân	Hệ thống										
1. Người quản lý truy cập danh sách người dùng	2. Hiển thị danh sách người dùng										
3. Người quản lý chọn người dùng để xem/chỉnh sửa thông tin	4. Hiển thị chi tiết thông tin người dùng										
5. Người quản lý cập nhật thông tin (nếu cần)	6. Lưu thông tin cập nhật										
7. Người quản lý cấp lại mật khẩu mới cho người dùng	8. Thông báo thành công và cập nhật mật khẩu mới										
Luồng thay đổi	Nếu có lỗi khi lưu thông tin hoặc cấp lại mật khẩu, hệ thống hiển thị thông báo lỗi										
Ngoại lệ	Người quản lý không có quyền truy cập; mất kết nối mạng khi thực hiện thao tác										

Bảng 2.6: Đặc tả use case Quản lý người dùng

2.4 Yêu cầu phi chức năng

- **Độ tin cậy:** Hệ thống cần hoạt động ổn định 24/7 với thời gian downtime tối thiểu, đảm bảo dữ liệu ra vào và thanh toán được ghi nhận chính xác và không bị mất mát.
- **Tính dễ sử dụng:** Giao diện web dành cho người dùng và quản lý phải trực quan, dễ hiểu, hỗ trợ truy cập trên nhiều thiết bị như máy tính, điện thoại di động.
- **Tính bảo trì và mở rộng:** Hệ thống được thiết kế theo kiến trúc mô-đun, dễ dàng nâng cấp, bảo trì và tích hợp thêm các tính năng mới trong tương lai.
- **Yêu cầu về cơ sở dữ liệu:** Sử dụng cơ sở dữ liệu quan hệ có khả năng xử lý

lượng dữ liệu lớn, sao lưu tự động và phục hồi khi có sự cố.

- **Yêu cầu bảo mật:** Bảo vệ thông tin người dùng, đảm bảo các giao dịch thanh toán an toàn, áp dụng các cơ chế xác thực và phân quyền truy cập hợp lý.
- **Yêu cầu về công nghệ:** Sử dụng các công nghệ phổ biến, hỗ trợ phát triển nhanh và triển khai trên đa nền tảng; hệ thống phần cứng sử dụng ESP32, cảm biến siêu âm và camera ESP để đảm bảo chi phí hợp lý.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương này trình bày chi tiết các công nghệ và nền tảng được sử dụng trong đồ án xây dựng hệ thống bãi đỗ xe thông minh. Mỗi công nghệ được phân tích theo hai tiêu chí: Giải quyết vấn đề/yêu cầu cụ thể nào đã nêu trong Chương 2; So sánh với các giải pháp thay thế để làm rõ lý do lựa chọn. Các công nghệ được lựa chọn bao gồm: thuật toán nhận diện YOLOv8, framework Django, cơ sở dữ liệu SQLite3 và các cảm biến, thiết bị phần cứng.

3.1 YOLOv8 – Nhận diện biển số xe

Ứng dụng trong hệ thống

YOLOv8 (You Only Look Once version 8) là một mô hình phát hiện đối tượng thời gian thực được sử dụng trong hệ thống để tự động nhận diện biển số xe khi phương tiện ra/vào bãi đỗ. Công nghệ này giải quyết trực tiếp yêu cầu “Tự động kiểm soát ra/vào bãi đỗ dựa trên nhận diện biển số” và liên quan đến use case *Quản lý ra/vào*. Việc sử dụng YOLOv8 giúp hệ thống tự động xác định xem phương tiện đã được đăng ký trên hệ thống chưa, từ đó quyết định cho phép mở cổng hoặc chuyển sang chế độ xác thực khác như thẻ RFID.

Lý do lựa chọn

YOLOv8 là phiên bản mới và mạnh mẽ của dòng mô hình YOLO, được phát triển bởi Ultralytics. Mô hình có kiến trúc nhẹ, tốc độ xử lý nhanh, khả năng phát hiện chính xác các đối tượng nhỏ như biển số xe trong điều kiện thực tế (ánh sáng thay đổi, nhiều xe ra vào liên tục). Ngoài ra, YOLOv8 hỗ trợ dễ dàng huấn luyện lại (retraining) với bộ dữ liệu tùy chỉnh và được triển khai thuận tiện bằng Python, phù hợp để tích hợp với backend của hệ thống sử dụng Django. Đây là yếu tố then chốt giúp đảm bảo hiệu suất xử lý thời gian thực và khả năng bảo trì hệ thống.

Kết luận

YOLOv8 được lựa chọn nhờ đạt được sự cân bằng tối ưu giữa tốc độ, độ chính xác và khả năng tích hợp. Bên cạnh đó, cộng đồng phát triển mạnh, tài liệu đầy đủ và hỗ trợ tốt cho việc tùy chỉnh mô hình cũng là những yếu tố quan trọng giúp nâng cao khả năng mở rộng và bảo trì hệ thống.



Hình 3.1: Hình ảnh YOLOv8

3.2 Django – Nền tảng phát triển Web

Ứng dụng trong hệ thống

Django là framework web mạnh mẽ của Python, dùng để xây dựng giao diện người dùng và quản trị. Django đáp ứng các yêu cầu trong mục 2.2 và 2.3 về chức năng: đăng ký tài khoản, nạp tiền, tra cứu lịch sử gửi xe, thống kê doanh thu và quản lý người dùng.

Lý do lựa chọn

Django hỗ trợ đầy đủ các tính năng web cần thiết: ORM, xác thực người dùng, quản trị giao diện, bảo mật form. Việc sử dụng cùng ngôn ngữ với YOLOv8 giúp đồng bộ hệ thống, giảm chi phí tích hợp.

Kết luận: Django được lựa chọn vì hỗ trợ đầy đủ tính năng, tích hợp tốt với YOLOv8, đồng thời phù hợp phát triển hệ thống nhanh chóng.



Hình 3.2: Hình ảnh django kết hợp python

3.3 SQLite3 – Hệ quản trị cơ sở dữ liệu

Ứng dụng trong hệ thống

SQLite3 được sử dụng để lưu trữ thông tin người dùng, lịch sử xe ra/vào, nạp tiền và thống kê doanh thu. Đây là cơ sở dữ liệu nhẹ, phù hợp với quy mô hệ thống

bãi đỗ xe vừa và nhỏ, như đã phân tích ở mục 2.1 và 2.3.5.

Lý do lựa chọn

SQLite tích hợp mặc định trong Django, dễ sử dụng, không cần thiết lập server riêng, phù hợp với hệ thống đơn người dùng hoặc mạng nội bộ nhỏ.

Kết luận: SQLite3 được lựa chọn vì đơn giản, đủ dùng, phù hợp với đề tài và giúp triển khai nhanh, ít lỗi.



Hình 3.3: Hình ảnh SQLite3

3.4 Cảm biến hồng ngoại LM393 – Phát hiện vật cản

Ứng dụng trong hệ thống

Cảm biến hồng ngoại LM393 được sử dụng tại cổng vào/ra bãi xe nhằm phát hiện sự hiện diện của xe đến gần. Khi xe tiến vào khu vực cảm biến, LM393 phát tín hiệu cho hệ thống điều khiển tự động kích hoạt camera và quy trình nhận diện biển số xe.

Cảm biến này đóng vai trò quan trọng trong việc đảm bảo việc ghi nhận thời điểm ra/vào được chính xác, phục vụ cho tính toán phí gửi xe và cập nhật lịch sử gửi xe trong hệ thống.

Lý do lựa chọn

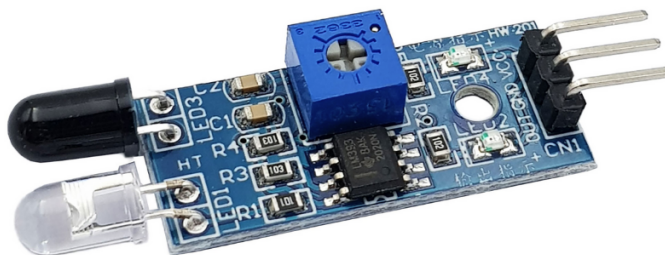
LM393 là loại cảm biến hồng ngoại có thiết kế đơn giản, độ trễ thấp và chi phí rẻ, dễ tích hợp với các board vi điều khiển như Arduino hoặc Raspberry Pi. Nó có khả năng phát hiện vật cản trong khoảng cách từ 2–30 cm.

So sánh các lựa chọn thay thế

- **Cảm biến siêu âm HC-SR04:** Có khả năng đo khoảng cách chính xác hơn, hoạt động trong khoảng xa hơn (2–400 cm). Tuy nhiên, giá thành cao hơn và phản hồi chậm hơn trong môi trường có nhiều nhiễu.
- **Cảm biến quang điện (photoelectric sensor):** Độ chính xác cao, nhưng thường dùng cho môi trường công nghiệp lớn, đòi hỏi hệ thống lắp đặt phức tạp hơn và chi phí cao.

Kết luận: LM393 là lựa chọn hợp lý trong dự án do đáp ứng tốt yêu cầu về phát

hiện vật cần trong phạm vi gần, chi phí thấp và dễ triển khai với các nền tảng vi điều khiển thông dụng.



Hình 3.4: Hình ảnh cảm biến hồng ngoại LM393

3.5 Module WiFi + BLE ESP32 NodeMCU LuaNode32

Ứng dụng trong hệ thống

Module ESP32 NodeMCU LuaNode32 là bộ vi điều khiển tích hợp WiFi và Bluetooth Low Energy (BLE), được sử dụng trong hệ thống để:

- Giao tiếp với cảm biến hồng ngoại để phát hiện phương tiện tại cổng.
- Điều khiển camera và gửi tín hiệu về hệ thống backend (Django server).
- Truyền dữ liệu nhận diện, thời điểm ra/vào và trạng thái cổng qua WiFi.

ESP32 là cầu nối giữa phần cứng (thiết bị nhúng) và phần mềm (hệ thống quản lý).

Lý do lựa chọn

ESP32 tích hợp cả WiFi lẫn BLE, có hiệu năng cao với giá thành thấp, rất phù hợp cho các ứng dụng IoT như bãi đỗ xe thông minh. Ngoài ra, nó hỗ trợ nhiều ngôn ngữ lập trình như Arduino, MicroPython, Lua và có cộng đồng hỗ trợ lớn.

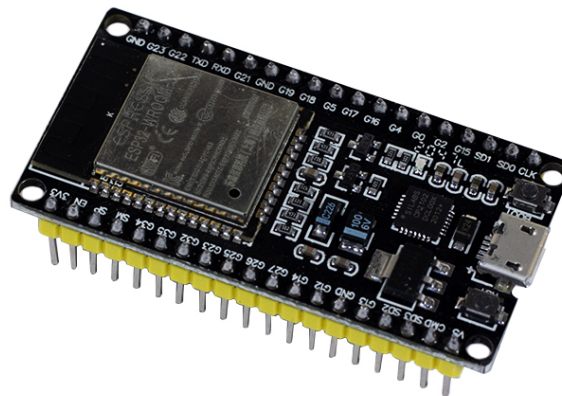
So sánh các lựa chọn thay thế

- **ESP8266:** Rẻ hơn và tiêu thụ điện năng thấp hơn, nhưng chỉ hỗ trợ WiFi, không có BLE, RAM thấp và khả năng xử lý yếu hơn so với ESP32.
- **Raspberry Pi Pico W:** Có hỗ trợ WiFi nhưng không có BLE. Cấu hình thấp

hơn và tài nguyên thư viện chưa phong phú bằng ESP32.

- **Arduino Uno + module WiFi rời (ESP01):** Cần thêm module WiFi ngoài, khiến kết nối kém ổn định, tiêu tốn nhiều chân IO, và không hỗ trợ BLE.

Kết luận: ESP32 NodeMCU LuaNode32 được lựa chọn vì có đủ năng lực xử lý, tích hợp sẵn WiFi/BLE, dễ lập trình và triển khai, đáp ứng tốt yêu cầu thu thập dữ liệu và truyền thông thời gian thực cho hệ thống bãi đỗ xe thông minh.



Hình 3.5: Hình ảnh ESP32 NodeMCU LuaNode32

3.6 Module RFID RC522 13.56MHz

Ứng dụng trong hệ thống

Module RFID RC522 được sử dụng để xác thực phương tiện chưa đăng ký biển số xe khi ra/vào bãi gửi. Người dùng sử dụng thẻ RFID để quét tại cổng vào/ra, từ đó:

- Kích hoạt mở cổng tự động nếu thẻ hợp lệ.
- Ghi nhận thời điểm vào/ra và tính phí gửi xe.
- Kết thúc phiên gửi xe và hiển thị mã QR để thanh toán nếu cần.

Lý do lựa chọn

RC522 là module RFID phổ biến, hoạt động ở tần số 13.56 MHz, có khả năng đọc thẻ nhanh và chính xác trong khoảng cách từ 2–5cm. Giá thành rẻ, dễ tích hợp với các vi điều khiển như ESP32. Ngoài ra, thư viện hỗ trợ phong phú giúp đơn giản hoá việc lập trình và tích hợp.

So sánh các lựa chọn thay thế

- **PN532:** Hỗ trợ cả giao tiếp I2C, SPI, UART và có thể đọc cả thẻ NFC. Tuy nhiên, chi phí cao hơn và cấu hình phức tạp hơn RC522.
- **MFRC500:** Cùng họ với RC522 nhưng ít được hỗ trợ cộng đồng, khó triển

khai với ESP32.

- **EM18 (125kHz):** Là RFID tần số thấp, khả năng bảo mật kém, tốc độ chậm, không tương thích với chuẩn MIFARE.

Kết luận: Với nhu cầu xác thực nhanh, giá rẻ, dễ lập trình và đủ bảo mật ở mức ứng dụng, RC522 là lựa chọn hợp lý nhất cho hệ thống kiểm soát vào/ra bằng RFID tại bãi đỗ xe.



Hình 3.6: Hình ảnh Module RFID RC522

3.7 Servo Tower Pro SG90 – Mô tơ điều khiển đóng/mở cổng

Ứng dụng trong hệ thống

Servo SG90 được sử dụng để điều khiển cơ chế đóng/mở cổng vào và cổng ra trong bãi gửi xe. Mỗi khi hệ thống xác thực biển số xe hoặc thẻ RFID thành công, servo sẽ quay một góc 90–180 độ để mở hoặc đóng barrier.

Việc sử dụng servo giúp:

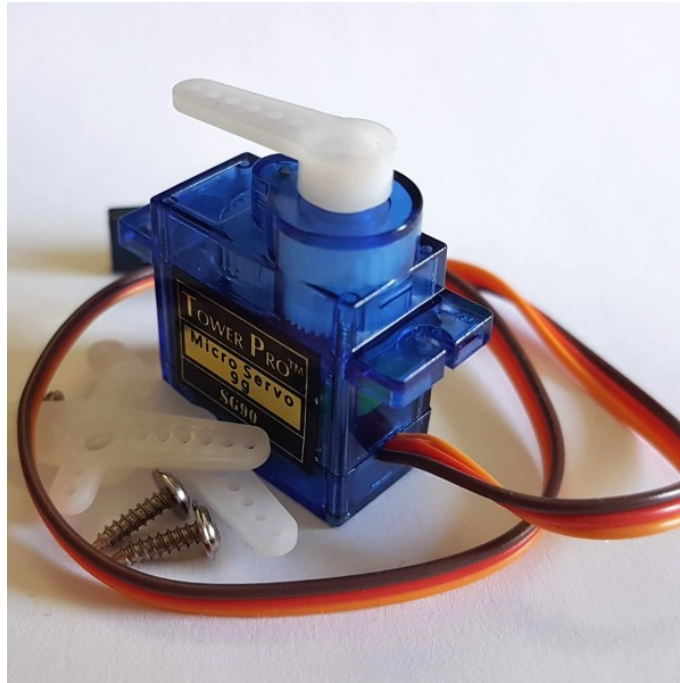
- Tự động hóa quá trình kiểm soát ra/vào.
- Giảm sự phụ thuộc vào nhân sự vận hành.
- Phản hồi nhanh, chính xác khi tích hợp với tín hiệu từ cảm biến/camera.

Lý do lựa chọn

Tower Pro SG90 là một trong những dòng servo phổ biến, có chi phí thấp và dễ sử dụng. Góc quay tối đa lên đến 180 độ, phù hợp cho cơ chế đóng/mở cổng nhỏ. Mô-tơ này có kích thước nhỏ gọn, phù hợp cho lắp đặt trong các hệ thống bãi xe

thông minh quy mô vừa và nhỏ.

Kết luận: Với yêu cầu điều khiển đơn giản, chi phí thấp và phản hồi tức thời cho cổng vào/ra quy mô nhỏ, Tower Pro SG90 là lựa chọn phù hợp và tối ưu trong hệ thống này.



Hình 3.7: Hình ảnh servo SG90

3.8 ESP32-CAM – Camera xử lý hình ảnh và truyền dữ liệu

Ứng dụng trong hệ thống

ESP32-CAM là module tích hợp camera và chip ESP32, được sử dụng để chụp ảnh biển số xe khi phương tiện ra/vào bãi giữ xe. Dữ liệu ảnh sau đó được truyền về hệ thống để mô hình YOLOv8 xử lý và nhận diện biển số.

Cụ thể, ESP32-CAM giúp giải quyết yêu cầu:

- Tự động chụp ảnh khi có phương tiện di chuyển qua cảm biến (mục tiêu từ Chương 2 – Quản lý ra/vào).
- Truyền hình ảnh về server qua Wi-Fi để phân tích thời gian thực.

Lý do lựa chọn

ESP32-CAM là một giải pháp toàn diện và giá rẻ cho các hệ thống nhưng có yêu cầu xử lý hình ảnh nhẹ. Với khả năng kết nối Wi-Fi tích hợp, thiết bị có thể hoạt động độc lập mà không cần thêm vi điều khiển trung gian.

- Tích hợp cả camera lẫn module Wi-Fi trên một bo mạch nhỏ gọn.
- Có thể lập trình bằng Arduino IDE, dễ triển khai.

- Giá thành rẻ, phù hợp với các hệ thống số lượng lớn.

So sánh các lựa chọn thay thế

- **Raspberry Pi + Camera Module:** Xử lý ảnh trực tiếp trên thiết bị mạnh hơn, chất lượng ảnh cao hơn nhưng giá cao, kích thước lớn và tiêu thụ điện năng nhiều.
- **OV7670 với Arduino:** Rất rẻ, nhưng không có kết nối Wi-Fi tích hợp, chất lượng ảnh kém hơn, khó triển khai thời gian thực.

Kết luận: Với yêu cầu chụp ảnh biển số đơn giản, gửi về server xử lý, ESP32-CAM là lựa chọn tối ưu về chi phí, tích hợp và khả năng kết nối trong hệ thống bãi giữ xe thông minh.



Hình 3.8: Hình ảnh ESP32 cam

3.9 Màn hình e-Ink 4.2 inch – Hiển thị trạng thái bãi đỗ và mã QR thanh toán

Ứng dụng trong hệ thống

Màn hình e-Ink 4.2 inch E042A13 A0 được sử dụng để hiển thị hai loại thông tin chính tại cổng ra/vào bãi gửi xe thông minh:

- **Tình trạng chỗ đỗ hiện tại:** cho biết ô đỗ tương ứng là CÓ XE hay TRỐNG, giúp người quản lý và người dùng dễ dàng quan sát.
- **Mã QR thanh toán:** khi xe ra về và tài khoản không đủ số dư, hệ thống sẽ hiển thị mã QR trên màn hình để người dùng quét và thực hiện thanh toán qua

ví điện tử.

Lý do lựa chọn

- Công nghệ e-Ink không tiêu tốn điện năng khi giữ trạng thái hiển thị – phù hợp với hệ thống IoT cần tiết kiệm điện, đặc biệt khi dùng pin hoặc năng lượng mặt trời.
- Màn hình không bị loá khi quan sát dưới ánh sáng ngoài trời, đảm bảo hiển thị rõ ràng cả ngày và đêm.
- Kích thước 4.2 inch đủ lớn để hiển thị mã QR có thể quét được dễ dàng bằng điện thoại.
- Tương thích tốt với ESP32 thông qua giao tiếp SPI, có nhiều thư viện mã nguồn mở như GxEPD hỗ trợ hiển thị văn bản và hình ảnh (QR code).

So sánh các công nghệ thay thế

- **OLED 0.96/1.3 inch:** nhỏ gọn, rẻ, nhưng quá nhỏ để hiển thị mã QR đầy đủ. Hiển thị kém dưới ánh sáng mạnh.
- **LED ma trận:** dễ gây chú ý, tốt để hiển thị thông báo ngắn, nhưng không thể hiện được QR code và thông tin chi tiết.

Kết luận: Màn hình e-Ink 4.2 inch là lựa chọn tối ưu cho hệ thống gửi xe thông minh cần hiển thị trạng thái đỗ và hỗ trợ thanh toán QR nhanh chóng, tiết kiệm điện năng và dễ triển khai trong môi trường ngoài trời.



Hình 3.9: Hình ảnh màn hình e-link 4.2 inch

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày quá trình thiết kế và triển khai hệ thống bãi đỗ xe thông minh, bao gồm việc tích hợp các phần mềm và phần cứng như nhận diện biển số, mở cổng tự động và thanh toán qua mã QR. Các công cụ, thư viện và kỹ thuật kiểm thử được sử dụng cũng sẽ được mô tả. Cuối cùng, kết quả triển khai và kiểm thử hệ thống sẽ được đánh giá để đảm bảo tính ổn định và hiệu quả của hệ thống.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Trong đề án này, tôi lựa chọn kiến trúc phần mềm **MVC (Model – View – Controller)** vì phù hợp với framework Django được sử dụng để phát triển hệ thống web quản lý bãi đỗ xe thông minh. Kiến trúc MVC giúp tách biệt rõ ràng giữa phần xử lý dữ liệu, giao diện và điều khiển, từ đó tăng tính linh hoạt, dễ bảo trì và mở rộng trong tương lai.

Giới thiệu sơ lược về kiến trúc MVC

Kiến trúc MVC bao gồm ba thành phần chính:

- **Model:** Đại diện cho lớp dữ liệu và xử lý nghiệp vụ. Model tương tác trực tiếp với cơ sở dữ liệu để lưu trữ và truy xuất thông tin.
- **View:** Chịu trách nhiệm hiển thị thông tin cho người dùng, thường được thể hiện qua giao diện web (HTML, CSS, JS).
- **Controller:** Là nơi xử lý các yêu cầu từ người dùng, điều phối giữa Model và View để đưa ra phản hồi phù hợp.

Django là một framework web Python theo mô hình MTV (Model – Template – View), trong đó:

- *Model* của Django chính là phần **Model** trong kiến trúc MVC.
- *View* của Django chính là phần **Controller** trong kiến trúc MVC.
- *Template* của Django đóng vai trò là **View** trong kiến trúc MVC.

Áp dụng kiến trúc MVC vào hệ thống

Trong hệ thống bãi đỗ xe thông minh, kiến trúc MVC được áp dụng như sau:

- **Model:** Bao gồm các lớp trong `models.py` như:
 - `Profile`: Lưu thông tin tài khoản người dùng.
 - `Camera`: Quản lý thông tin về camera.

- `AccessHistory`: Ghi nhận lịch sử gửi xe.
- `TopUpHistory`: Lưu thông tin nạp tiền và thanh toán.
- **View (Template)**: Gồm các file HTML, CSS, JS trong thư mục `templates/` dùng để:
 - Hiển thị giao diện đăng ký, đăng nhập.
 - Tra cứu lịch sử gửi xe.
 - Thống kê doanh thu và bảng điều khiển quản lý.
- **Controller (View trong Django)**: Các hàm xử lý trong `views.py` đảm nhiệm:
 - Xử lý yêu cầu từ người dùng.
 - Tương tác với model để truy xuất dữ liệu.
 - Điều hướng người dùng đến các giao diện thích hợp.

Điều chỉnh và mở rộng trong kiến trúc thực tế

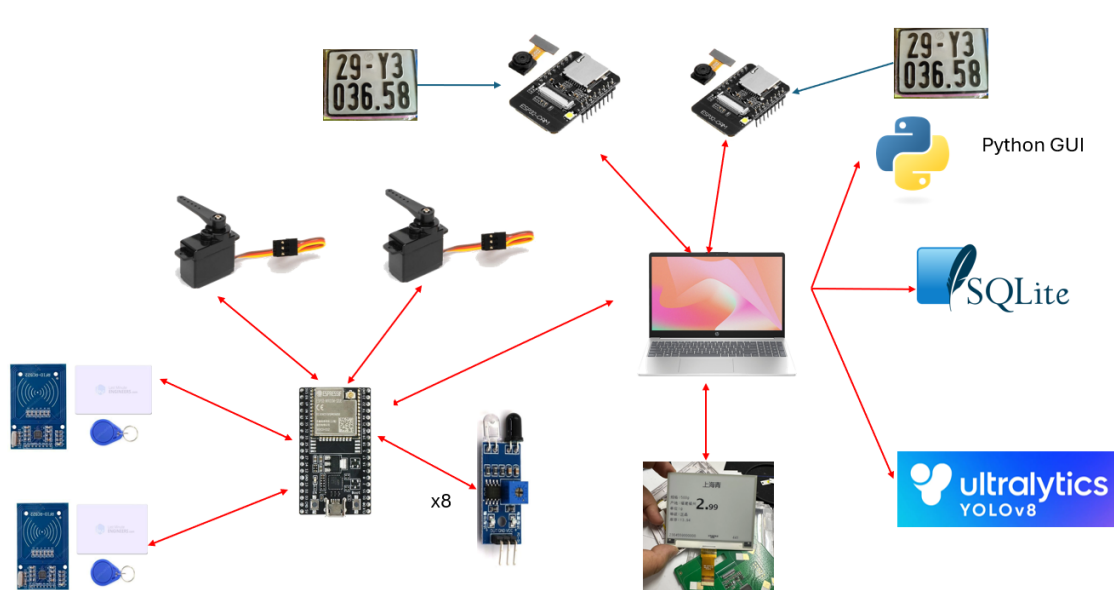
Hệ thống được mở rộng thêm một số thành phần ngoài kiến trúc MVC truyền thống như:

- **Module YOLOv8**: Tích hợp vào lớp Controller như một dịch vụ xử lý ảnh, nhận diện biển số xe và gửi kết quả về backend để ghi nhận vào database.
- **Thiết bị IoT (ESP32-CAM, RFID, cảm biến)**: Giao tiếp thông qua các API REST do Django cung cấp. Các tín hiệu được xử lý tại controller và phản hồi theo thời gian thực.
- **WebSocket**: Sử dụng để cập nhật trạng thái gửi xe, giúp giao diện người dùng nhận dữ liệu theo thời gian thực mà không cần tải lại trang.

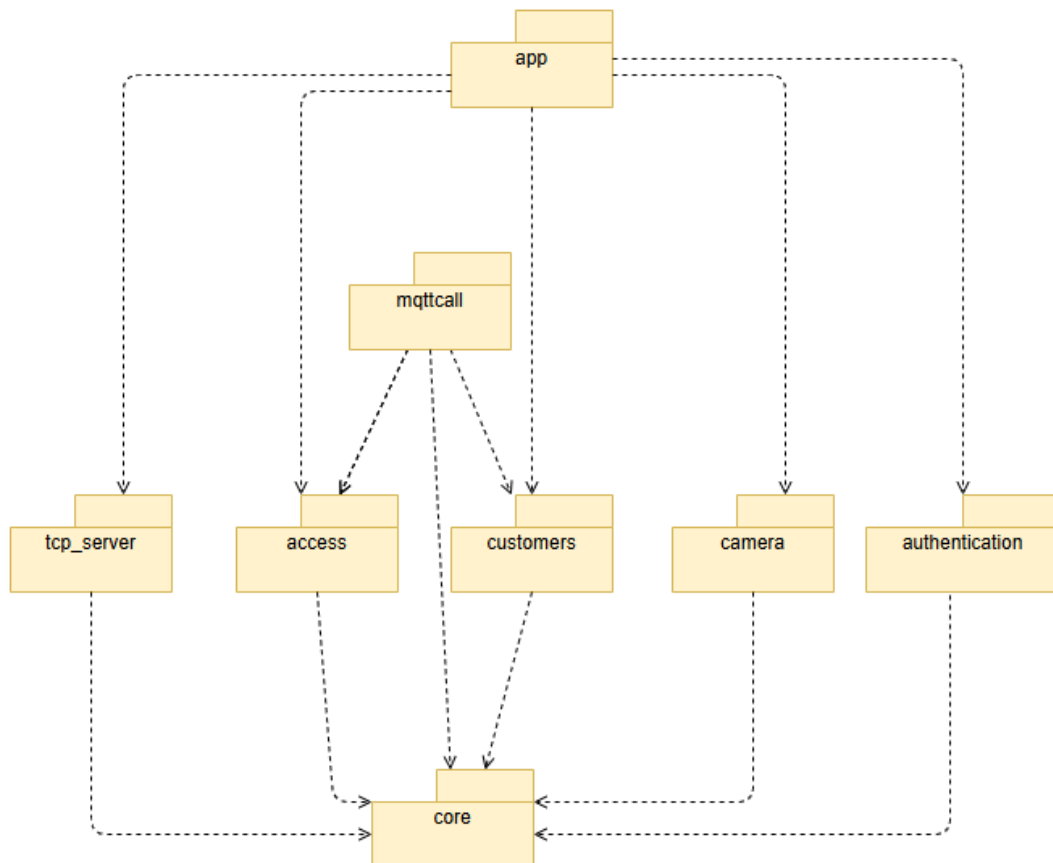
Kết luận

Kiến trúc MVC phù hợp với định hướng phát triển phần mềm hiện đại, giúp phân tách rõ ràng các lớp chức năng trong hệ thống. Khi kết hợp với Django – một framework hỗ trợ MVC mạnh mẽ – hệ thống đạt được hiệu quả cao trong xử lý, dễ dàng mở rộng, bảo trì và tích hợp với các công nghệ AIoT khác.

4.1.2 Thiết kế tổng quan



Hình 4.1: Sơ đồ tổng quan thiết kế của hệ thống



Hình 4.2: Mô hình thiết kế tổng quan phần mềm

Các gói chính và vai trò

app: Gói trung tâm điều phối (main application layer) – điều khiển luồng logic chính, gọi đến các module khác.

core: Chứa file cấu hình các gói khác.

access: Quản lý lịch sử ra vào.

customers: Quản lý dữ liệu khách hàng: thông tin hồ sơ, lịch sử nạp tiền.

camera: Tích hợp camera giám sát, chụp hình, phát hiện hình ảnh.

authentication: Xác thực người dùng, đăng ký, đăng nhập.

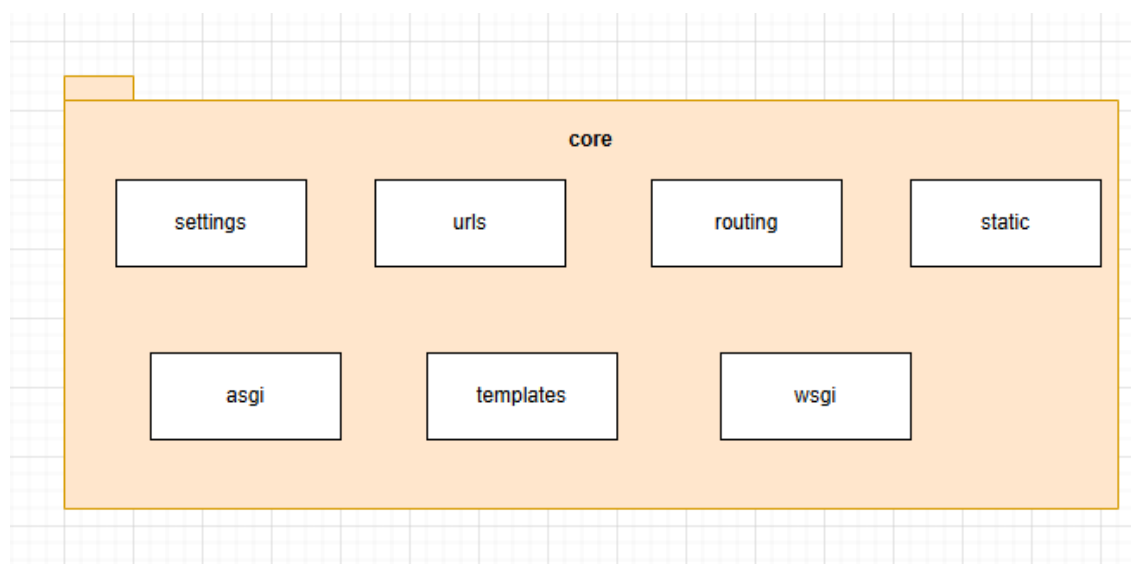
mqttcall: Giao tiếp qua giao thức MQTT – phổ biến trong hệ thống IoT.

tcp_server: Cung cấp hoặc tiếp nhận kết nối qua TCP, có thể dùng để liên lạc thiết bị hoặc server nội bộ.

4.1.3 Thiết kế chi tiết gói

Gói core – Cấu hình trung tâm hệ thống

Gói `core` đóng vai trò là thành phần trung tâm của hệ thống, chịu trách nhiệm cấu hình toàn cục cho toàn bộ ứng dụng Django. Đây là nơi tích hợp các ứng dụng con, định nghĩa các luồng định tuyến (URL), xử lý WebSocket và hỗ trợ kết nối thời gian thực



Hình 4.3: Thiết kế chi tiết gói core

• Các thành phần chính trong gói:

- `settings.py` – Cấu hình toàn cục Django: đăng ký apps, middleware, static/media path, DB, REST framework, JWT, Redis,...

- `urls.py` – Định tuyến URL chính của toàn hệ thống, kết nối các ứng dụng như `app`, `authentication`, `access`, v.v.
- `routing.py` – Xử lý định tuyến WebSocket, thường sử dụng Redis làm lớp trung gian truyền thông.
- `consumers.py` – Xử lý luồng giao tiếp WebSocket với frontend, ví dụ cập nhật thời gian thực tình trạng xe vào/ra.
- `asgi.py` – Entry point cho ASGI server (Daphne/Uvicorn), hỗ trợ cả HTTP và WebSocket.
- `wsgi.py` – Entry point cho WSGI server truyền thống (Gunicorn).
- `templates/` – Chứa các template HTML chung của hệ thống.
- `static/`, `staticfiles/` – Tài nguyên frontend (CSS, JS, icon), được collect về từ các app.

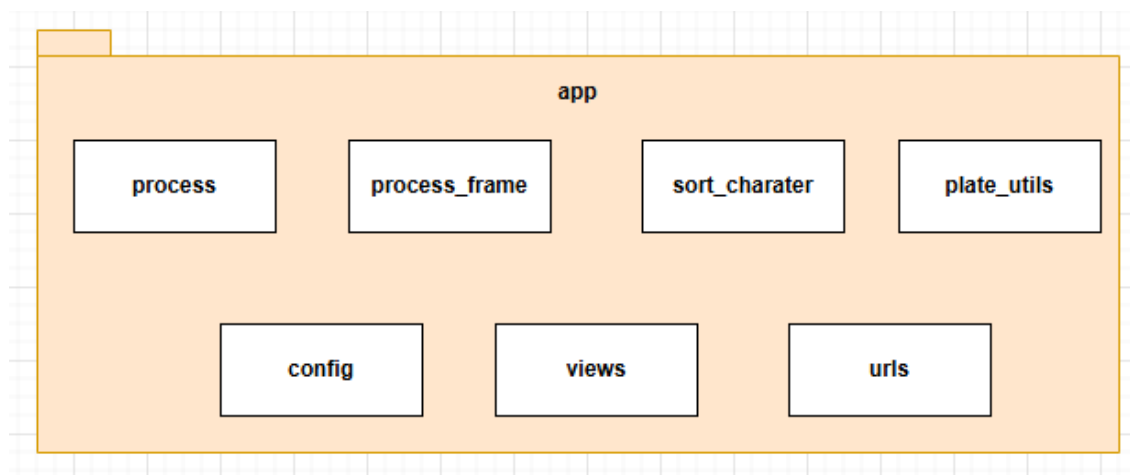
• Môi quan hệ phụ thuộc:

- Gọi tới tất cả các gói con trong hệ thống (như `camera`, `access`, `customers`,...).
- Sử dụng Redis, Channels và ASGI để hỗ trợ kết nối WebSocket cho luồng camera và điều khiển thiết bị.

Tóm lại, `core` là gói "gắn kết hệ thống", đóng vai trò như phần khung tổ chức, giúp các ứng dụng nhỏ bên trong hoạt động hài hòa theo cấu trúc đã định nghĩa sẵn.

Gói `app` – Xử lý nghiệp vụ chính và điều phối giao diện

Gói `app` là một trong những thành phần quan trọng nhất trong hệ thống, chịu trách nhiệm xử lý các chức năng nghiệp vụ chính như: nhận diện biển số xe, phân tích ảnh từ camera, xác định thời gian đỗ xe, tính phí gửi xe, và hiển thị thông tin lên dashboard. Ngoài ra, `app` còn điều phối hiển thị các trang web chính của người dùng.



Hình 4.4: Thiết kế chi tiết gói app

• **Các thành phần chính trong gói:**

- `views.py` – Xử lý các request đến từ người dùng, render template và hiển thị dữ liệu cần thiết.
- `urls.py` – Định tuyến các URL chính như `‘/home/‘`, `‘/lich-su-ra-vao/‘`, `‘/ho-so/‘`, `‘/doanh-thu/‘`, v.v.
- `process.py` – Hàm tiền xử lý ảnh, tách ký tự biển số.
- `process_frame.py` – Xử lý frame từ luồng camera (từng khung hình).
- `sort_charater.py` – Sắp xếp ký tự theo vị trí trái-phải sau khi nhận dạng.
- `function/plate_utils.py` – Xử lý ảnh, phân tích vùng ảnh biển số, hỗ trợ tách ký tự từ YOLOv8.
- `config.py` – Cấu hình cục bộ liên quan đến ứng dụng.

• **Mối quan hệ phụ thuộc:**

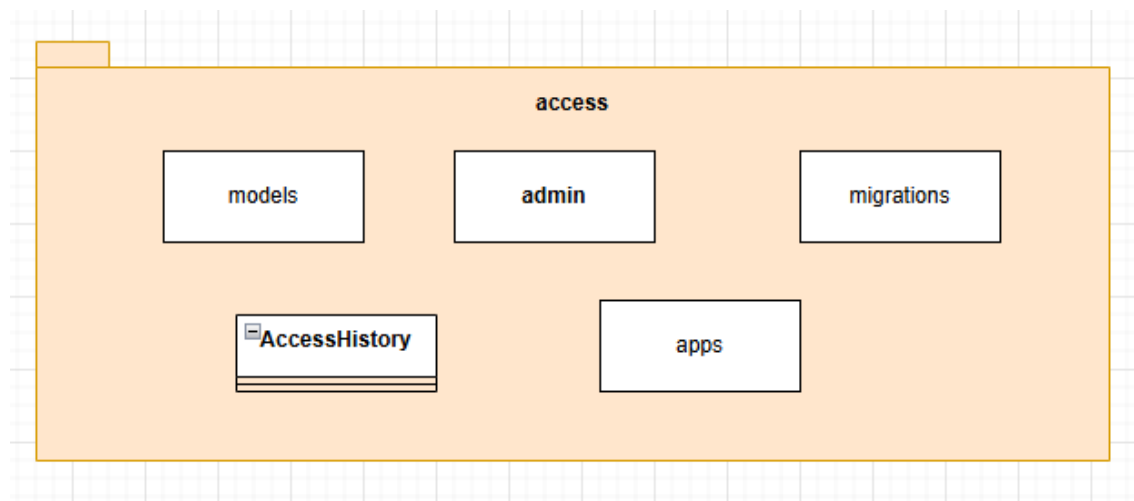
- Phụ thuộc `access.models`, `customers.models`, `camera.models`, `authentication.models` để lấy dữ liệu lịch sử, hồ sơ người dùng, ảnh.
- Dùng chung các tiện ích từ `‘core.settings‘` như cấu hình media, static, WebSocket,...
- Gọi hàm từ `‘tcp_server/function/plate_utils.py‘` khi xử lý ảnh nhận diện từ ảnh chụp trực tiếp.

Tóm lại, gói app là trung tâm hiển thị và điều phối chính trong hệ thống, đóng vai trò liên kết giữa người dùng – dữ liệu – mô hình AI. Mọi luồng tương tác giao diện

đều đi qua gói này.

Gói **access** – Quản lý lượt gửi xe và lịch sử ra/vào

Gói **access** đảm nhiệm vai trò ghi nhận và xử lý các lượt gửi xe của người dùng. Đây là gói cốt lõi để phục vụ nghiệp vụ theo dõi, tra cứu, tính toán thời gian gửi xe và phí gửi tương ứng. Mọi phương tiện đi qua cổng vào/ra đều được ghi nhận tại đây, bất kể sử dụng biển số hay thẻ RFID.



Hình 4.5: Thiết kế chi tiết gói **access**

• Các thành phần chính trong gói:

- `models.py` – Định nghĩa model `AccessHistory` để lưu thông tin truy cập của người dùng.
- `admin.py` – Cho phép quản lý các bản ghi `AccessHistory` từ trang admin Django.
- `apps.py` – Cấu hình khởi tạo app `access`.
- `migrations.py` – Lịch sử các thay đổi database schema liên quan đến `AccessHistory`.

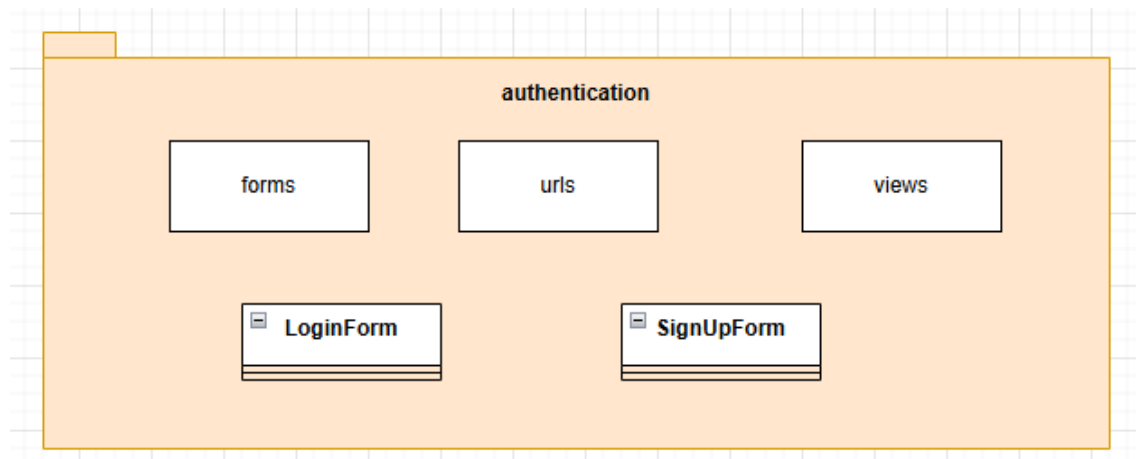
• Mối quan hệ phụ thuộc:

- Phụ thuộc gói `customers` để liên kết người dùng với tài khoản.
- Nhận dữ liệu từ `camera` hoặc `tcp_server` để xử lý ảnh, mã RFID.
- Dữ liệu hiển thị được truy xuất từ gói app (dashboard, tra cứu).

Tóm lại, gói **access** là trung tâm ghi nhận toàn bộ dữ liệu vận hành của bãi xe thông minh, đảm bảo tính minh bạch và theo dõi chi tiết cho từng lượt gửi xe.

Gói `authentication` – Xác thực và phân quyền người dùng

Gói `authentication` chịu trách nhiệm xử lý các chức năng liên quan đến xác thực, phân quyền và quản lý đăng ký, đăng nhập cho hệ thống bãi đỗ xe thông minh. Đây là gói cần thiết giúp phân biệt giữa người dùng hệ thống (admin, khách hàng) và hạn chế quyền truy cập theo vai trò.



Hình 4.6: Thiết kế chi tiết gói `authentication`

- **Các thành phần chính trong gói:**

- `views.py` – Xử lý các request từ người dùng liên quan đến xác thực (login, logout...)
- `forms.py` – Cung cấp biểu mẫu đăng ký `SignUpForm`, đăng nhập `LoginForm` người dùng sử dụng Django forms.
- `urls.py` – Khai báo các endpoint như `/login/`, `/logout/`, `/register/`.

- **Mối quan hệ phụ thuộc:**

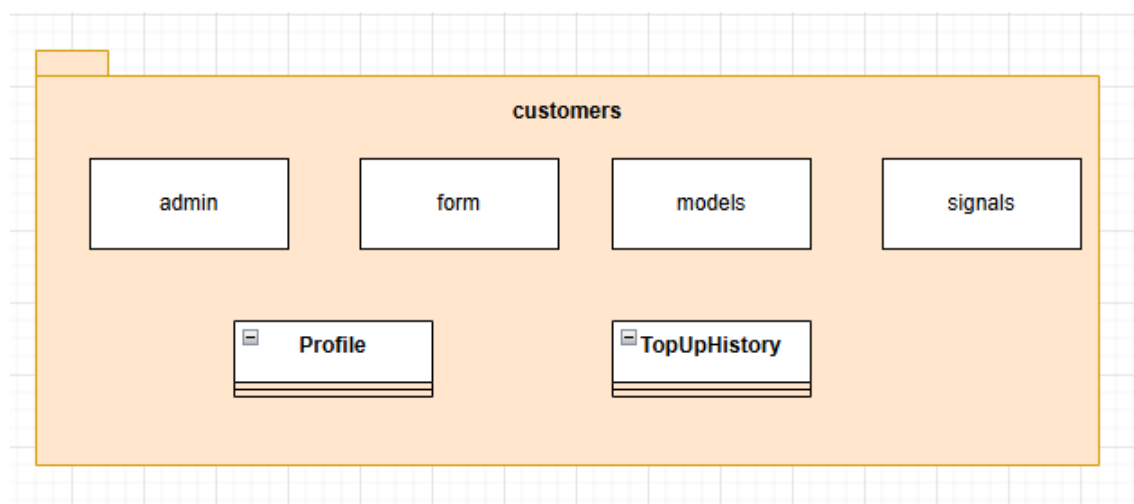
- Sử dụng mô hình người dùng mở rộng của Django (`'AbstractUser'` hoặc `'User'` mặc định).
- Dữ liệu người dùng liên kết với bảng hồ sơ từ gói `customers`.
- Giao diện điều hướng và hiển thị quyền truy cập được xử lý từ gói `app`.

Gói `authentication` tuy nhỏ gọn nhưng đóng vai trò quan trọng trong bảo mật hệ thống. Mọi tương tác với dashboard, API, và chức năng quản lý đều đi qua bước xác thực và phân quyền tại đây.

Gói `customers` – Quản lý thông tin người dùng và hồ sơ cá nhân

Gói `customers` là nơi lưu trữ và xử lý các thông tin liên quan đến người dùng của hệ thống. Gói này cung cấp khả năng cập nhật hồ sơ cá nhân, liên kết giữa tài khoản người dùng với biển số xe, đồng thời quản lý số dư ví điện tử của người

dùng.



Hình 4.7: Thiết kế chi tiết gói customers

- **Các thành phần chính trong gói:**

- `models.py` – Khai báo model Profile và TopUpHistory. Là nơi lưu dữ liệu người dùng và giao dịch.
- `forms.py` – Biểu mẫu cập nhật hồ sơ cá nhân, nạp tiền vào ví.
- `views.py` – Xử lý các chức năng:
- `admin.py` – Cho phép quản lý Profile, TopUpHistory trên trang admin.
- `signals.py` – Tự động tạo Profile khi user được khởi tạo

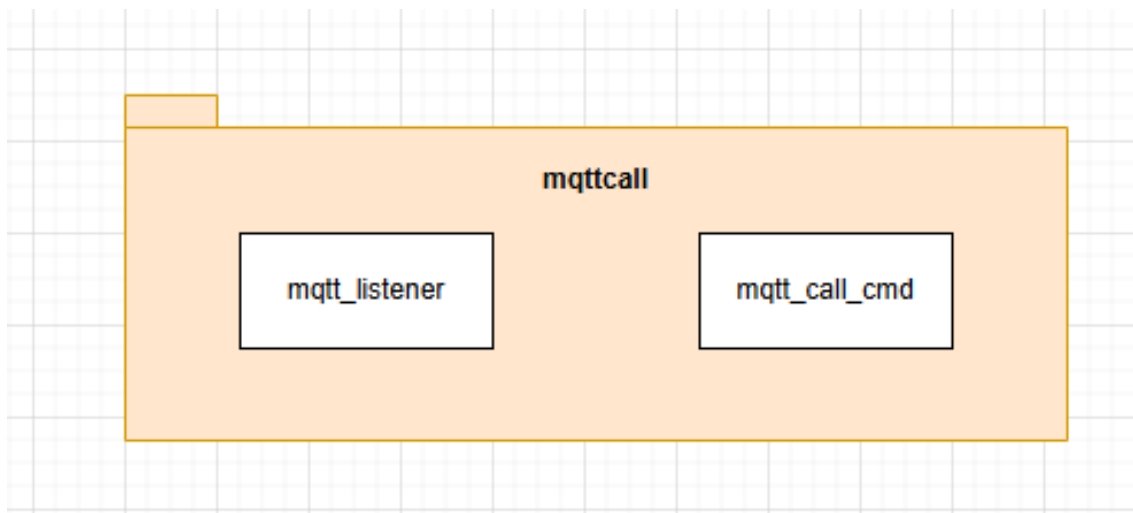
- **Mối quan hệ phụ thuộc:**

- Phụ thuộc `authentication.models` để liên kết với tài khoản người dùng.
- Kết nối với gói `access` để hỗ trợ tra cứu lịch sử gửi xe của từng người.

Tóm lại, `customers` là gói chịu trách nhiệm lưu trữ thông tin người dùng một cách đầy đủ và chính xác, đóng vai trò làm cầu nối giữa tài khoản hệ thống và thực thể sử dụng dịch vụ gửi xe.

Gói `mqtcall`

Gói `mqtcall` là nơi xử lý giao dịch nạp tiền và thanh toán thông qua nội dung bản tin MQTT và gửi sự kiện thời gian thực đến giao diện web thông qua WebSocket và điều khiển mở cổng qua TCP.



Hình 4.8: Thiết kế chi tiết gói mqttcall

• **Các thành phần chính trong gói:**

– `mqtt_listener.py`

* `on_connect()` – Thiết lập kết nối và đăng ký các chủ đề MQTT cần thiết.

* `on_message()` – Tiếp nhận bản tin MQTT và đưa vào hàng đợi xử lý.

* `process_mqtt_queue()` – Phân tích nội dung bản tin, thực hiện nạp tiền hoặc xử lý thanh toán lỗi ra.

* `broadcast_access_event()` – Gửi sự kiện WebSocket tới frontend để cập nhật giao diện thời gian thực.

* `start_mqtt_listener()` – Khởi động toàn bộ quy trình MQTT và xử lý đa luồng.

– `mqtt_call_cmd.py` - Khởi động MQTT listener thông qua dòng lệnh

• **Mối quan hệ phụ thuộc:**

– `customers.models` – Dùng để cập nhật số dư và trạng thái giao dịch nạp tiền.

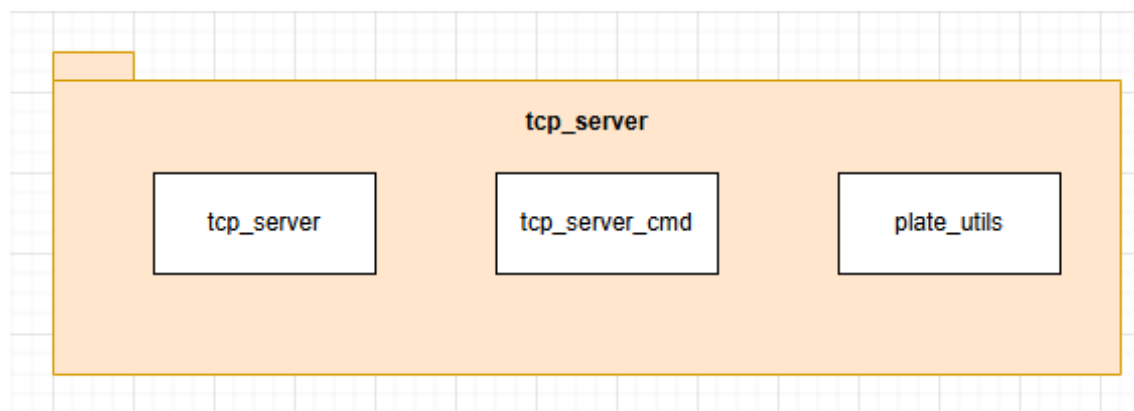
– `access.models` – Ghi nhận lịch sử xe ra và lưu số tiền thanh toán.

– `asgiref.sync, channels.layers` – Gửi dữ liệu thời gian thực qua WebSocket.

– `core.settings` – Lưu trữ thông tin cấu hình MQTT broker (host, port, topic, tài khoản).

Gói `tcp_server` – Giao tiếp mạng TCP với thiết bị phần cứng

Gói `tcp_server` tạo một TCP server lắng nghe kết nối đến từ các thiết bị cổng hoặc dịch vụ khác trong hệ thống nội bộ, là một tầng trung gian giao tiếp vật lý (Gửi lệnh điều khiển cổng) giữa hệ thống và phần cứng qua kết nối TCP.



Hình 4.9: Thiết kế chi tiết gói `tcp_server`

- **Các thành phần chính trong gói:**

- `tcp_server.py` – Tạo socket server, lắng nghe và xử lý gói dữ liệu gửi từ thiết bị.
- `tcp_server_cmd` – Lệnh CLI để khởi động TCP server qua `manage.py`.
- `plate_utils.py` – Công cụ xử lý biến số

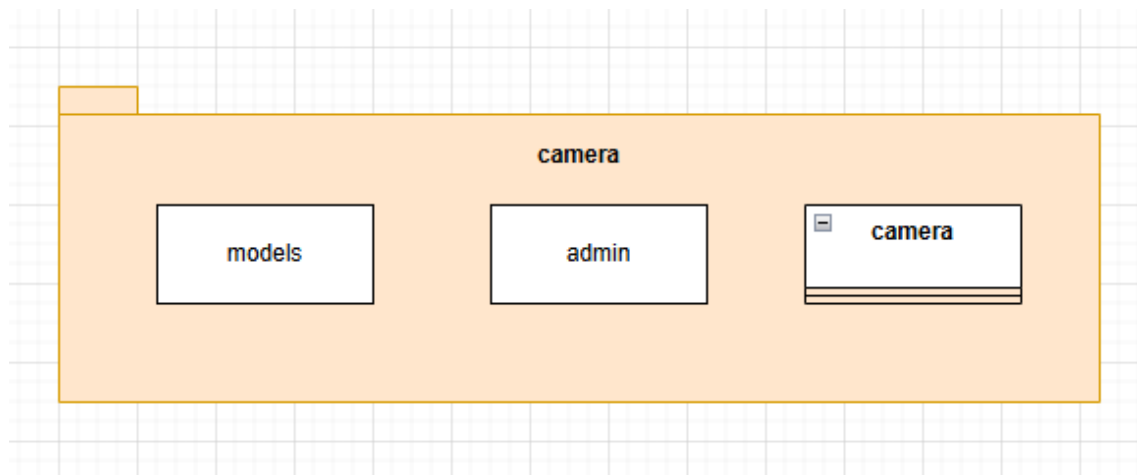
- **Mối quan hệ phụ thuộc:**

- Phụ thuộc `core` hỗ trợ cấu hình port, xử lý thời gian, log.

Gói `tcp_server` cho phép giao tiếp mạng mức thấp giữa thiết bị IoT và phần mềm, rất phù hợp với camera gửi dữ liệu thời gian thực (ảnh, tín hiệu).

Gói `camera`

Gói `camera` quản lý thông tin cấu hình các thiết bị camera (ESP32-CAM) được lắp tại cổng vào và cổng ra, phân biệt từng camera theo tên, vị trí lắp đặt và trạng thái hoạt động.



Hình 4.10: Thiết chi tiết gói camera

- **Các thành phần chính trong gói:**

- `models.py`

- * Camera – Mô hình lưu thông tin thiết bị:

- `camera_name`: Tên định danh thiết bị.
 - `stream_url`: Đường dẫn stream video (nếu có).
 - `camera_type`: Phân loại entry (vào) hoặc exit (ra).
 - `is_active`: Trạng thái hoạt động (bật/tắt).
 - `created_at, updated_at`: Dấu thời gian phục vụ log hệ thống.

- `admin.py`

- * Cấu hình để hiển thị và thao tác trên mô hình Camera qua giao diện quản trị Django.

- **Mối quan hệ phụ thuộc:**

- phụ thuộc `core` để hỗ trợ cấu hình camera.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện người dùng của hệ thống được thiết kế hướng tới nền tảng web, sử dụng bộ giao diện Volt Bootstrap Dashboard của AppSeed với độ phân giải màn hình tối ưu là từ 1366x768 trở lên, hỗ trợ đầy đủ dải màu RGB 24-bit.

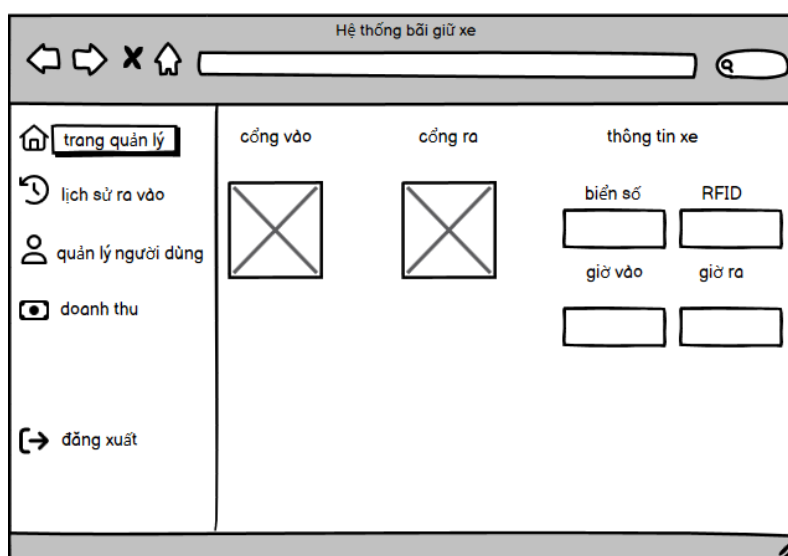
Nguyên tắc thiết kế

Giao diện được xây dựng với tiêu chí đơn giản, trực quan, dễ thao tác cho cả người dùng và người quản trị, cụ thể:

- **Màu sắc:** Chủ đạo là gam màu xanh navy (#1f2d3d) phối với nền sáng, đảm bảo tính tương phản cao.
- **Kiểu nút:** Các nút chức năng như *Đăng xuất*, *Cập nhật*, *Lọc*, *Nạp tiền*... sử dụng màu nổi bật (đỏ, xanh lá) với bo góc nhẹ.
- **Hiển thị thông báo:** Các cảnh báo quan trọng (vd. không có camera) sử dụng thẻ màu cam/đỏ và biểu tượng tương ứng giúp người dùng dễ nhận diện.
- **Bố cục:** Giao diện chia thành 2 vùng: thanh điều hướng bên trái cố định và vùng hiển thị nội dung động bên phải.

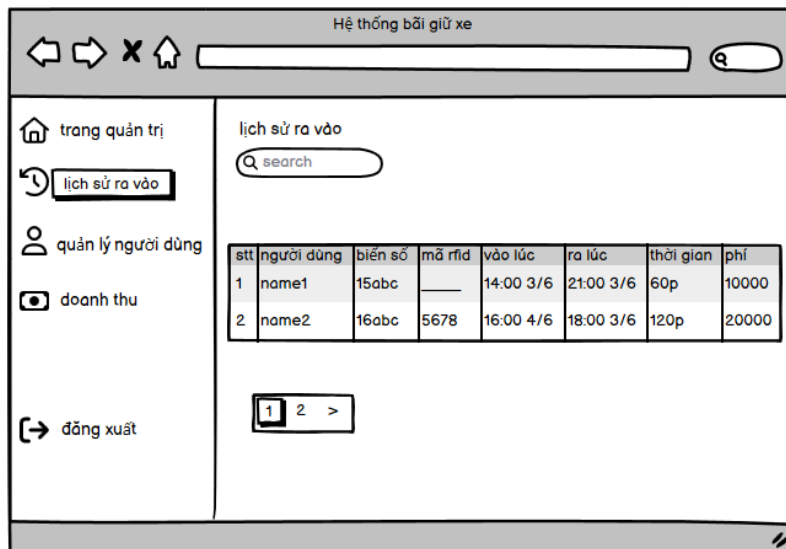
Các chức năng chính

- **Màn hình trang chủ quản trị:** Hiển thị trạng thái camera cổng ra/vào và thông tin phương tiện gần nhất.



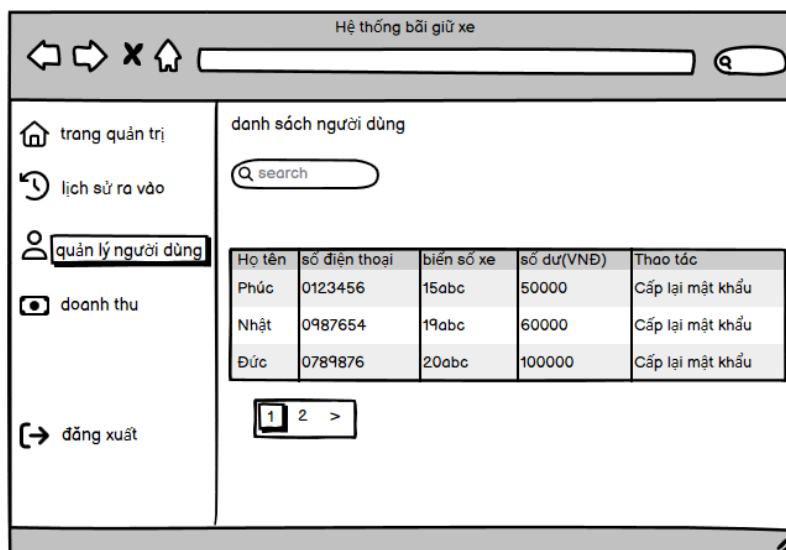
Hình 4.11: Màn hình minh họa trang quản trị

- **Màn hình lịch sử ra vào:** Cho phép lọc theo thời gian, biển số, mã RFID, loại khách, và xuất dữ liệu chi tiết.



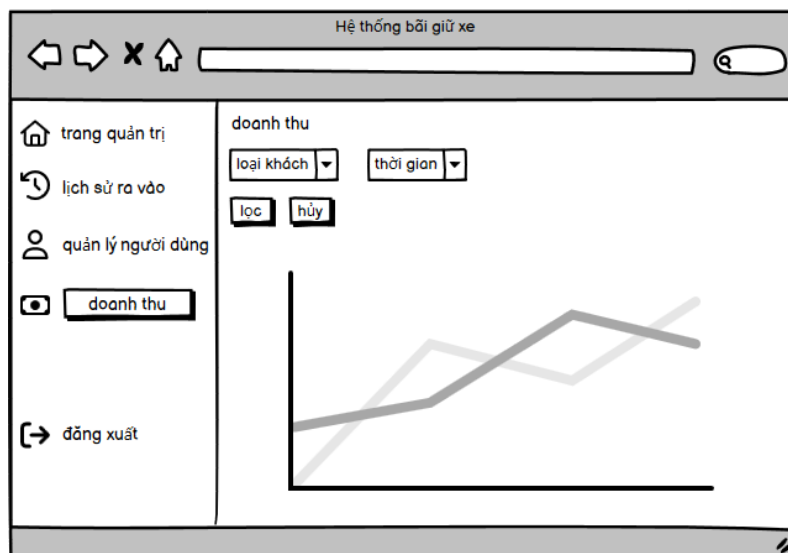
Hình 4.12: Màn hình hình họa lịch sử ra vào

- **Màn hình quản lý người dùng:** Hiển thị danh sách, thông tin tài khoản, hỗ trợ chỉnh sửa và cấp lại mật khẩu.



Hình 4.13: Màn hình minh họa quản lý người dùng

- **Màn hình thống kê doanh thu:** Hiển thị biểu đồ đường theo ngày/tháng và tổng doanh thu.



Hình 4.14: Màn hình minh họa thống kê doanh thu

- **Màn hình hồ sơ người dùng:** Cập nhật thông tin cá nhân, theo dõi số dư và lịch sử nạp tiền.

stt	số tiền	thời gian	trạng thái
1	10000	15:00 3/6	thành công
2	5000	16:00 4/6	thành công

Hình 4.15: Màn hình minh họa hồ sơ người dùng

Kết luận

Thiết kế giao diện được lựa chọn theo hướng hiện đại, dễ sử dụng và phù hợp với mọi đối tượng, đảm bảo tính khả dụng trong môi trường triển khai thực tế tại

bãi xe thông minh.

Lưu ý: Các hình ảnh trên chỉ là thiết kế giao diện (mockup) phục vụ trình bày ý tưởng, không đại diện cho sản phẩm cuối cùng. Giao diện có thể thay đổi trong quá trình phát triển và triển khai thực tế.

4.2.2 Thiết kế lớp

Trong phần này, chúng tôi trình bày thiết kế chi tiết cho bốn lớp quan trọng nhất trong hệ thống: **Profile**, **TopUpHistory**, **AccessHistory** và **Camera**. Đây là các lớp cốt lõi chịu trách nhiệm lưu trữ và xử lý dữ liệu nghiệp vụ trong hệ thống bãi đỗ xe thông minh.

Lớp Profile

Bảng 4.1: Thuộc tính lớp Profile

Tên thuộc tính	Kiểu dữ liệu	Mô tả
user	OneToOneField(User)	Liên kết với tài khoản người dùng
full_name	CharField	Họ tên người dùng
phone	CharField	Số điện thoại
license_plate	CharField	Biển số xe (tự động viết hoa)
balance	DecimalField	Số dư tài khoản

Phương thức:

- `save()` – tự động viết hoa biển số trước khi lưu.

Lớp TopUpHistory

Bảng 4.2: Thuộc tính lớp TopUpHistory

Tên thuộc tính	Kiểu dữ liệu	Mô tả
user	ForeignKey(User)	Người dùng thực hiện giao dịch
amount	DecimalField	Số tiền được nạp
timestamp	DateTimeField	Thời điểm giao dịch
status	CharField	Trạng thái: pending, success, failed

Phương thức:

- `__str__()` – trả về chuỗi dạng “username - amount - trạng thái”.

Lớp AccessHistory

Bảng 4.3: Thuộc tính lớp AccessHistory

Tên thuộc tính	Kiểu dữ liệu	Mô tả
uid	CharField	Mã truy cập duy nhất (UUID rút gọn)
user	ForeignKey(User)	Người dùng tương ứng
license_plate	CharField	Biển số xe
rfid_code	CharField	Mã thẻ RFID
check_in	DateTimeField	Thời gian vào
check_out	DateTimeField	Thời gian ra
fee	IntegerField	Phí đỗ xe

Phương thức:

- `save()` – tự động tạo uid nếu chưa có.
- `get_license_plate_display()` – lấy biển số từ hồ sơ người dùng.
- `duration_minutes()` – tính thời gian dừng xe theo phút.

Lớp Camera

Bảng 4.4: Thuộc tính lớp Camera

Tên thuộc tính	Kiểu dữ liệu	Mô tả
camera_name	CharField	Tên của thiết bị camera
stream_url	URLField	Đường dẫn đến video stream (RT-SP/HTTP)
camera_type	CharField	Loại camera: entry hoặc exit
is_active	BooleanField	Trạng thái hoạt động
created_at	DateTimeField	Thời điểm thêm vào hệ thống
updated_at	DateTimeField	Thời điểm cập nhật cuối

Phương thức:

- `__str__()` – hiển thị tên + loại camera theo định dạng: "Tên (Cổng ra/vào)".

4.2.3 Thiết kế cơ sở dữ liệu

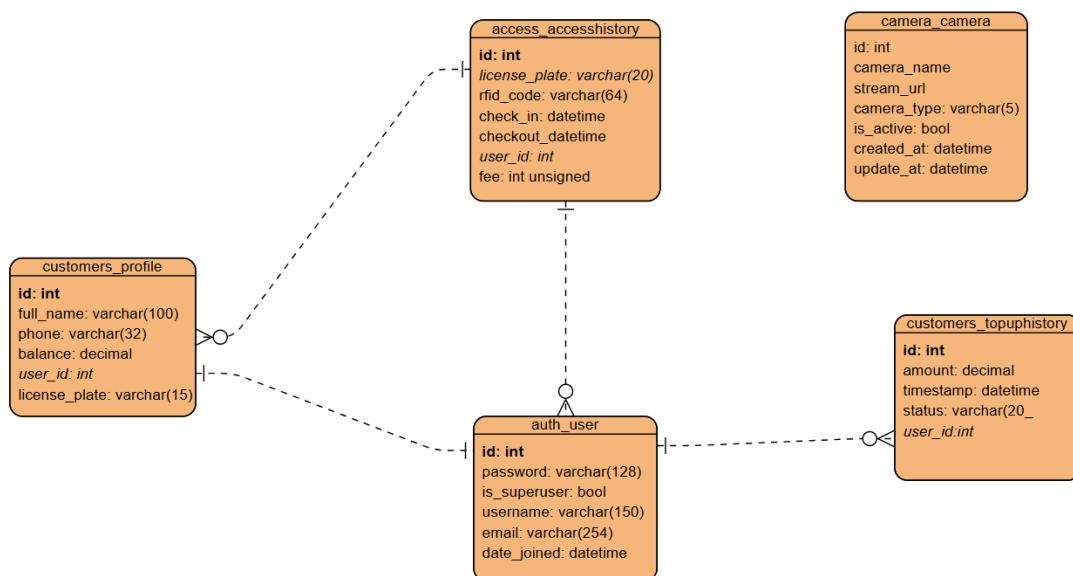
Phần này trình bày thiết kế cơ sở dữ liệu cho hệ thống, bao gồm việc thiết kế và giải thích biểu đồ thực thể liên kết (E-R diagram) mô tả các thực thể và mối quan hệ giữa chúng. Biểu đồ này giúp hiểu rõ cấu trúc của dữ liệu trong hệ thống và cách các thực thể tương tác với nhau.

Trong hệ thống của tôi, các thực thể chính bao gồm **Người dùng (auth_user)**,

Hồ sơ khách hàng (customers_profile), Lịch sử nạp tiền (customers_topuphistory), và Lịch sử ra vào bãi xe (access_accesshistory). Mỗi quan hệ giữa các thực thể này thể hiện cách thức người dùng có thể nạp tiền vào tài khoản, thực hiện các giao dịch và ra vào bãi xe.

- **auth_user** và **customers_profile**: Mỗi người dùng có thể có một hoặc nhiều hồ sơ khách hàng, do đó mỗi quan hệ giữa chúng là một-nhiều (1-n). Trường `user_id` trong bảng `customers_profile` liên kết với `id` trong bảng `auth_user`.
- **customers_profile** và **access_accesshistory**: Mỗi hồ sơ khách hàng có thể có nhiều lần vào/ra bãi xe, do đó mỗi quan hệ giữa chúng là một-nhiều (1-n). Trường `license_plate` trong bảng `access_accesshistory` liên kết với `license_plate` trong bảng `customers_profile`.
- **auth_user** và **customers_topuphistory**: Mỗi người dùng có thể thực hiện nhiều giao dịch nạp tiền, do đó mỗi quan hệ giữa chúng là một-nhiều (1-n). Trường `user_id` trong bảng `customers_topuphistory` liên kết với `id` trong bảng `auth_user`.

Dựa trên biểu đồ E-R, tôi sẽ thiết kế cơ sở dữ liệu trong hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) sử dụng SQL, trong đó mỗi bảng sẽ chứa các trường tương ứng với các thuộc tính của thực thể trong hệ thống. Cấu trúc bảng sẽ đảm bảo tính toàn vẹn dữ liệu và hỗ trợ các phép toán truy vấn hiệu quả.



Hình 4.16: Biểu đồ thực thể liên kết

4.2.4 Thiết kế phần cứng

Bộ điều khiển trung tâm ESP32, màn hình E-Ink 4.2"

Hệ thống sử dụng một vi điều khiển ESP32 làm trung tâm điều phối, đồng thời kết nối với các linh kiện ngoại vi như cảm biến RFID, mô tơ Servo, các nút nhấn và màn hình e-paper 3 màu. Mục đích là để xử lý tín hiệu từ thiết bị ra/vào, hiển thị trạng thái bãi xe và tương tác với server.

Cấu hình phần cứng tổng thể

Bảng 4.5: Sơ đồ kết nối phần cứng với ESP32

Thiết bị	Chân kết nối ESP32	Chức năng
RFID Reader 1	SS: D4, RST: D0	Đọc thẻ tại cổng vào
RFID Reader 2	SS: D5, RST: D0	Đọc thẻ tại cổng ra
Servo 1	D21	Điều khiển barrier tại cổng vào
Servo 2	D22	Điều khiển barrier tại cổng ra
Cảm biến hồng ngoại 1	D17	Xác định phương tiện tại cổng vào
Cảm biến hồng ngoại 2	D16	Xác định phương tiện tại cổng ra
Cảm biến hồng ngoại 3	D15	Xác định trạng thái slot đỗ xe #1
Cảm biến hồng ngoại 4	D13	Xác định trạng thái slot đỗ xe #2
Cảm biến hồng ngoại 5	D14	Xác định trạng thái slot đỗ xe #3
Cảm biến hồng ngoại 6	D25	Xác định trạng thái slot đỗ xe #4
Cảm biến hồng ngoại 7	D26	Xác định trạng thái slot đỗ xe #5
Cảm biến hồng ngoại 8	D27	Xác định trạng thái slot đỗ xe #6

Bảng 4.6: Kết nối giữa ESP32 và màn hình E-Paper 4.2 inch

Chân ESP32	Tín hiệu	Mô tả
GPIO13	SCLK	Clock SPI
GPIO14	MOSI	Gửi dữ liệu SPI
GPIO15	CS	Chip Select
GPIO27	DC	Data/Command: phân biệt lệnh và dữ liệu
GPIO26	RST	Reset màn hình
GPIO25	BUSY	Trạng thái BUSY (đang xử lý)

Chức năng điều khiển và hiển thị

Giao tiếp mạng:

- ESP32 kết nối Wi-Fi đến server để gửi sự kiện và nhận phản hồi TCP/HTTP.
- Phản hồi dạng "open_in", "open_out" được xử lý để điều khiển servo.

Xử lý tín hiệu và hiển thị:

- Khi nhận chuỗi dạng code :xxxxxx, ESP32 phân tích và:

- Nếu là `.bmp` → hiển thị ảnh QR trên e-paper (qua HTTP).
- Nếu là chuỗi nhị phân 6 bit → hiển thị trạng thái slot xe (có/không có xe).
- Màn hình e-paper 4.2 inch dùng thư viện `GxEPD2_3C` hỗ trợ 3 màu và chế độ tiết kiệm điện `hibernate()`.

Tích hợp với hệ thống phần mềm:

- ESP32 gửi sự kiện cổng ra/vào đến API backend tại `/api/gate/event/`.
- Dữ liệu nhận dạng từ RFID hoặc sensor sẽ được đồng bộ hoá với hệ thống lưu trữ.
- Mã QR sẽ được tạo từ server và gửi về dưới dạng chuỗi tải ảnh hoặc chuỗi xác định slot.

Kết luận

Sự kết hợp giữa ESP32 và màn hình e-paper giúp hệ thống có thể vừa điều khiển vật lý, vừa phản hồi trực quan cho người dùng. Đây là nền tảng quan trọng giúp xây dựng một mô hình IoT bãi giữ xe thông minh.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Sinh viên liệt kê các công cụ, ngôn ngữ lập trình, API, thư viện, IDE, công cụ kiểm thử, v.v. mà mình sử dụng để phát triển ứng dụng. Mỗi công cụ phải được chỉ rõ phiên bản sử dụng. SV nên kẻ bảng mô tả tương tự như Bảng 4.7. Nếu có nhiều nội dung trình bày, sinh viên cần xoay ngang bảng.

Bảng 4.7: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ / Thư viện	Địa chỉ URL
IDE lập trình Frontend và Backend	Visual Studio Code 1.99.3	https://code.visualstudio.com/
IDE lập trình ESP32	Arduino IDE 2.3.6	https://www.arduino.cc/en/software
Lập trình ứng dụng	Django 4.2.13	https://www.djangoproject.com/
API và RESTful service	Django REST Framework	https://www.django-rest-framework.org/
Database	SQLite3	https://sqlite.org/
Web server production	gunicorn 20.1.0, whitenoise 5.3.0	https://gunicorn.org/ , https://whitenoise.evans.io/
Giao tiếp MQTT	paho-mqtt	https://pypi.org/project/paho-mqtt/
WebSocket	Channels 4.2.0, Daphne 4.1.2, channels_redis 4.2.1	https://channels.readthedocs.io/
Xử lý ảnh	OpenCV-Python, Pillow	https://opencv.org/ , https://python-pillow.org/
Nhận diện biển số	YOLOv8 (Ultralytics)	https://github.com/ultralytics/ultralytics
Cấu hình môi trường	python-decouple 3.4, Unipath 1.1	https://pypi.org/project/python-decouple/ , https://pypi.org/project/Unipath/
Kiểm tra style code	pycodestyle 2.7.0	https://pypi.org/project/pycodestyle/
Import/Export dữ liệu	django-import-export	https://django-import-export.readthedocs.io/
Đồng bộ thời gian	pytz 2021.1	https://pypi.org/project/pytz/
Quản lý phiên bản mã nguồn	Git 2.34.1	https://git-scm.com/

4.3.2 Kết quả đạt được

Ứng dụng bãi đỗ xe thông minh của tôi đã hoàn thiện với các sản phẩm đóng gói phần mềm và phần cứng. Dưới đây là các thành phần chính của sản phẩm:

- **Phần mềm (Backend):** Hệ thống sử dụng Django và Django REST Framework để xây dựng API cho việc giao tiếp với frontend và các thiết bị phần cứng.
- **Phần mềm (Frontend):** Giao diện web đơn giản được xây dựng bằng HTML, CSS và JavaScript thuần.
- **Phần cứng:** Các thiết bị phần cứng bao gồm ESP32, RFID reader, camera, servo và màn hình E-Ink, giúp hỗ trợ nhận diện xe, mở cổng và hiển thị thông tin.

Dưới đây là các thông tin thống kê về ứng dụng của tôi:

Mục	Số lượng
Tổng số dòng mã nguồn	8000 dòng
Tổng số lớp	4 lớp
Tổng số gói phần mềm	8 gói
Dung lượng toàn bộ mã nguồn	125 MB
Dung lượng mã nguồn backend	119 MB
Dung lượng mã nguồn frontend	5 MB
Dung lượng phần mềm nhúng (ESP32)	1 MB
Số lượng database tables	5 bảng

Bảng 4.8: Thông tin thống kê ứng dụng

4.3.3 Ý nghĩa và vai trò của các thành phần

- **Backend (Django):** Cung cấp API để xử lý các yêu cầu từ frontend và các thiết bị phần cứng. Backend còn đảm nhận việc xác thực người dùng, tính phí dịch vụ, lưu trữ lịch sử xe vào/ra, v.v.
- **Frontend:** Cung cấp giao diện đơn giản cho người dùng để theo dõi tình trạng bãi đỗ xe, quét mã QR thanh toán, và yêu cầu mở cổng.
- **ESP32 và phần cứng:** Điều khiển các thiết bị vật lý như servo để mở cổng, camera nhận diện biển số xe, và RFID reader để quét thẻ.

4.3.4 Kết quả đạt được

- **Tự động hóa quy trình gửi xe:** Hệ thống đã hoàn thành việc tự động nhận diện biển số hoặc quét thẻ RFID, mở cổng, tính phí và thanh toán qua mã QR.
- **Giảm thời gian xử lý:** Quá trình vào/ra bãi đỗ xe đã được rút ngắn xuống còn dưới 5 giây, giúp tăng cường trải nghiệm người dùng.
- **Hệ thống hoạt động ổn định:** Hệ thống đã hoạt động ổn định trong nhiều giờ liên tục, không có sự cố nghiêm trọng về mạng hay phần cứng.

4.3.5 Minh họa các chức năng chính

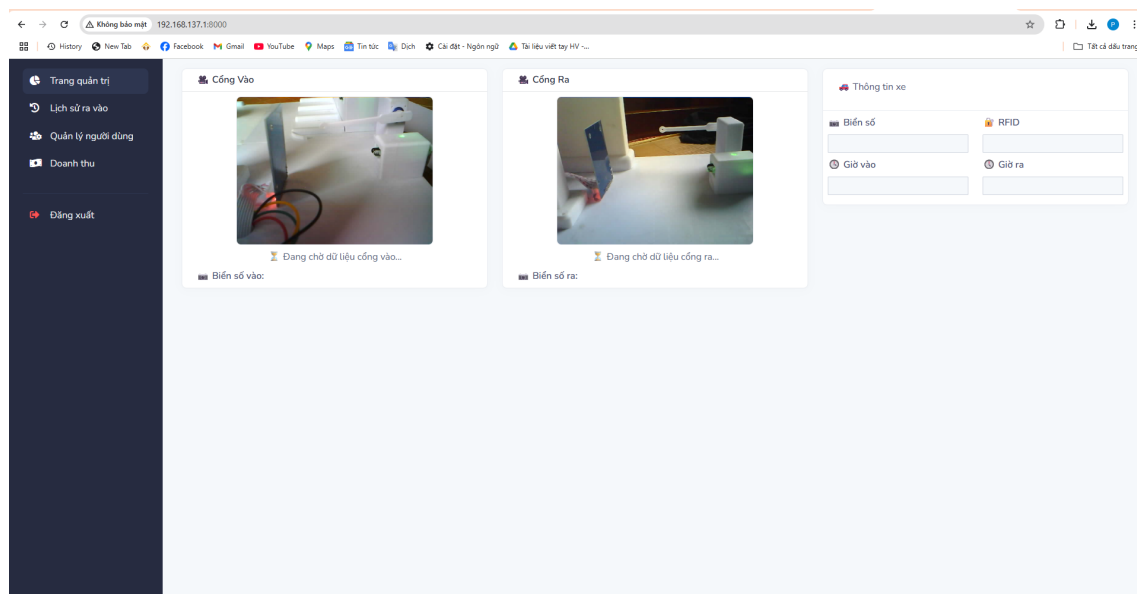
Màn hình người dùng cập nhật thông tin

Người dùng nhập các thông tin như họ tên, số điện thoại và biển số xe. Nếu nhập trùng số điện thoại hoặc biển số xe thì hệ thống sẽ báo lỗi. Người dùng chọn Nạp tiền, sau đó nhập số tiền (lớn hơn 5000 VNĐ), hệ thống sẽ hiện ra mã QR. Người dùng quét mã và thanh toán thành công, hệ thống sẽ cập nhật số dư tài khoản.

Hình 4.17: Màn hình cập nhật thông tin

Màn hình quản lý ra vào

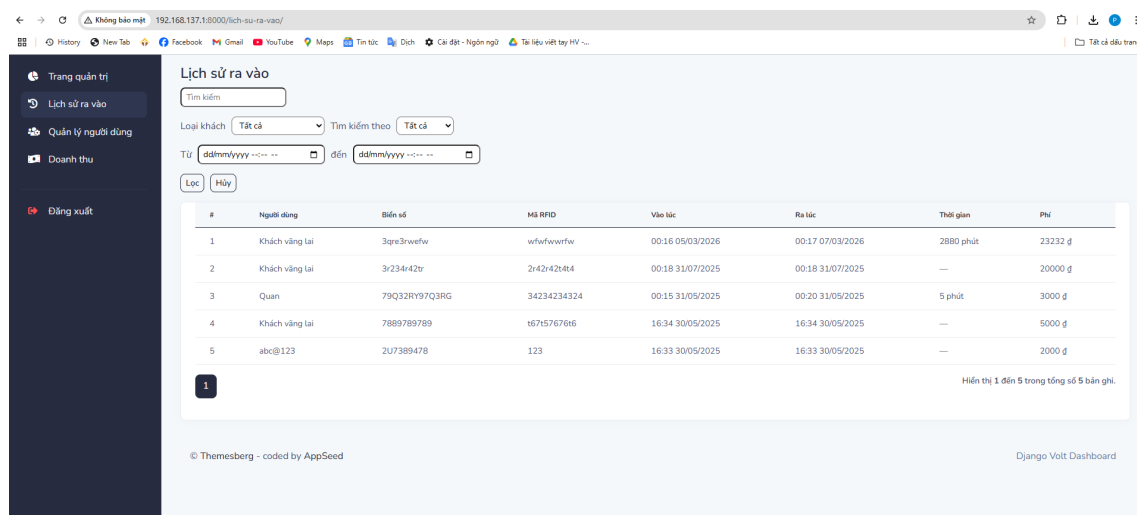
Ở màn hình này, hệ thống sẽ hiện ra hình ảnh truyền về từ ESP32 cam, khi có xe đi vào, sẽ nhận diện biển và hiển thị thông tin lên màn hình. Nếu biển xe đã có trong cơ sở dữ liệu, servo sẽ mở ra cho người dùng đi vào. Nếu biển xe chưa được đăng ký, hệ thống sẽ yêu cầu quét RFID để mở cổng vào.



Hình 4.18: Màn hình quản lý ra vào

Màn hình thống kê lịch sử ra vào

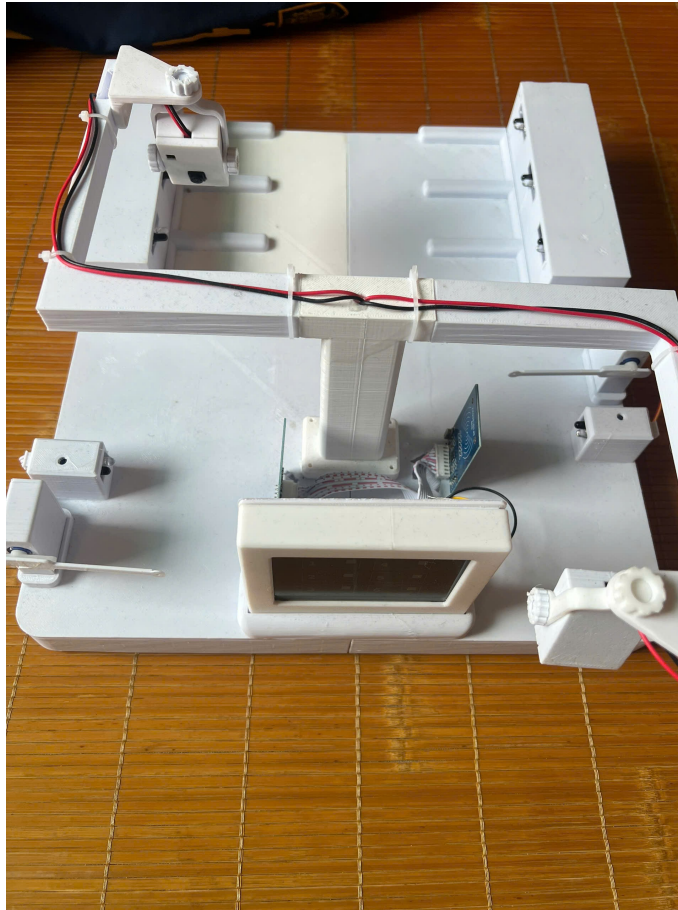
Người quản lý có thể theo dõi lịch sử ra vào bao gồm người dùng, biển xe, thời gian ra/vào, thời gian đỗ và phí. Ngoài ra, người quản lý có thể tìm kiếm theo biển xe, lọc theo loại khách hàng và thời gian (ngày, tháng, năm hoặc trong khoảng thời gian cụ thể)



Hình 4.19: Màn hình thống kê lịch sử ra vào

Hình ảnh phần cứng

Phần cứng được xây dựng và in 3d bằng bìa fomex. Các thiết bị được kết nối và đi dây ở mặt dưới. Bộ nguồn Adapter 12V 3A kết hợp với module hạ áp DC XL4015 5V để cấp điện cho các thiết bị khác.



Hình 4.20: Hình ảnh phần cứng của hệ thống

4.4 Kiểm thử

Chức năng nhận diện biển số và mở cổng, cũng như chức năng thanh toán tự động qua mã QR là hai tính năng cốt lõi của hệ thống bãi đỗ xe thông minh.

4.4.1 Nhận diện biển số và mở cổng

STT	Mô tả trường hợp thử nghiệm	Các bước kiểm tra	Kết quả
1	Nhận diện biển số xe đã tồn tại và mở cổng	1. Xe đến gần cổng 2. Camera nhận diện biển số xe 3. Kiểm tra biển số trong cơ sở dữ liệu 4. Nếu biển số hợp lệ, cổng tự động mở	Cổng mở tự động khi biển số tồn tại và được xác thực thành công

STT	Mô tả trường hợp thử nghiệm	Các bước kiểm tra	Kết quả
2	Nhận diện biển số chưa tồn tại	1. Xe đến gần cổng 2. Camera nhận diện biển số xe 3. Kiểm tra biển số trong cơ sở dữ liệu 4. Nếu biển số không tồn tại, hệ thống yêu cầu quét thẻ RFID	Hệ thống yêu cầu người dùng quét thẻ RFID
3	Quét thẻ RFID hợp lệ và mở cổng	1. Người dùng quét thẻ RFID 2. Hệ thống kiểm tra tính hợp lệ của thẻ 3. Nếu thẻ hợp lệ, cổng tự động mở	Cổng mở tự động khi thẻ RFID hợp lệ
4	Quét thẻ RFID không hợp lệ	1. Người dùng quét thẻ RFID 2. Hệ thống kiểm tra tính hợp lệ của thẻ 3. Nếu thẻ không hợp lệ, hệ thống hiển thị thông báo lỗi	Hệ thống hiển thị thông báo lỗi và yêu cầu quét lại thẻ

4.4.2 Thanh toán tự động qua mã QR

STT	Mô tả trường hợp thử nghiệm	Các bước kiểm tra	Kết quả
1	Người dùng có đủ tiền trong tài khoản	1. Người dùng rời khỏi bãi đỗ xe 2. Hệ thống tính toán thời gian đỗ và phí dịch vụ 3. Hệ thống trừ tiền từ tài khoản người dùng 4. Cổng tự động mở khi thanh toán thành công	Cổng mở tự động khi người dùng có đủ tiền và thanh toán thành công
2	Người dùng không đủ tiền trong tài khoản	1. Người dùng rời khỏi bãi đỗ xe 2. Hệ thống tính toán thời gian đỗ và phí dịch vụ 3. Nếu không đủ tiền, hệ thống hiển thị mã QR để thanh toán 4. Người dùng quét mã QR và thực hiện thanh toán	Hệ thống hiển thị mã QR và chờ thanh toán thành công
3	Thanh toán qua mã QR thành công	1. Người dùng thanh toán qua mã QR 2. Hệ thống kiểm tra trạng thái thanh toán 3. Nếu thanh toán thành công, cổng tự động mở	Cổng mở tự động khi thanh toán qua mã QR thành công
4	Thanh toán qua mã QR không thành công	1. Người dùng thanh toán qua mã QR 2. Hệ thống kiểm tra trạng thái thanh toán 3. Nếu thanh toán không thành công, hệ thống hiển thị thông báo lỗi	Hệ thống hiển thị thông báo lỗi khi thanh toán không thành công

4.4.3 Tổng kết kiểm thử

- Tổng số trường hợp kiểm thử: 8 trường hợp
- Trường hợp kiểm thử đạt yêu cầu: 8/8
- Các lỗi không đạt yêu cầu (nếu có) có thể do phần cứng (camera, RFID reader), lỗi kết nối mạng trong quá trình thanh toán hoặc lỗi server MQTT.

4.5 Triển khai

Hệ thống nhận diện biển số xe thông minh được triển khai theo mô hình tích hợp giữa phần cứng nhúng (ESP32-CAM, cảm biến siêu âm, servo, RFID) và phần mềm xử lý ảnh (YOLOv8 kết hợp OpenCV). Quá trình thử nghiệm được tiến hành trong môi trường giả lập với các thành phần cụ thể như sau:

Mô hình triển khai thử nghiệm

- **Thiết bị phần cứng:**
 - Camera ESP32-CAM dùng để chụp ảnh biển số xe tại khu vực vào và ra.
 - Cảm biến siêu âm kết nối với vi điều khiển ESP32 để xác định trạng thái từng vị trí đỗ xe.
 - Servo module điều khiển đóng/mở cổng khi hệ thống xác nhận biển số hợp lệ.
- **Hệ thống xử lý trung tâm:**
 - Server xử lý hình ảnh và nhận diện biển số được triển khai trên máy tính cá nhân với cấu hình như sau:
 - * CPU: Intel Core i7-11800H @ 2.3GHz
 - * RAM: 16GB
 - * GPU: NVIDIA GeForce RTX 3050Ti
 - * Hệ điều hành: Windows 11
 - Giao diện quản lý được xây dựng bằng HTML, Css, Js cho phép giám sát và tương tác trực tiếp với hệ thống.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Phát hiện và nhận diện biển số xe

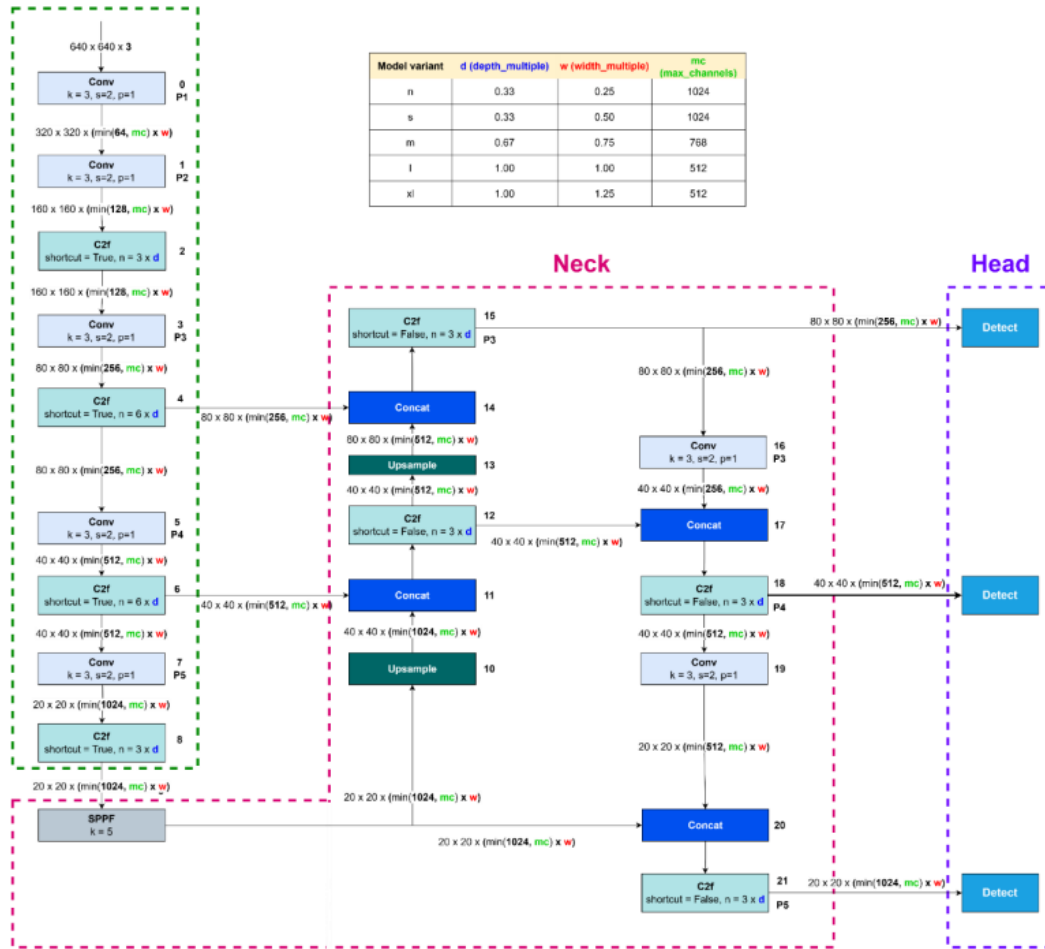
5.1.1 Giới thiệu về bài toán/ vấn đề

Phát hiện và nhận diện biển số xe là một bài toán trong lĩnh vực thị giác máy tính, có rất nhiều ứng dụng đặc biệt là trong hệ thống quản lý bãi đỗ xe thông minh. Bài toán yêu cầu xác định vị trí biển số xe trên hình ảnh và nhận diện chính xác các ký tự trong biển số để phục vụ cho các mục đích như nhận diện biển xe, nhận diện người dùng và tính phí ra vào tự động.

5.1.2 Giải pháp

Trong các phương pháp hiện có, YOLO (You Only Look Once) đã nổi bật như một thuật toán phát hiện đối tượng nhanh chóng và chính xác, giúp tối ưu hóa quá trình nhận diện biển số xe. Với khả năng xử lý ảnh theo thời gian thực và phát hiện đối tượng trong một lần duy nhất, YOLOv8 mang lại những cải tiến vượt trội trong việc phát hiện các đối tượng có kích thước nhỏ như biển số xe.

Trong phần tiếp theo, tôi sẽ tìm hiểu về YOLOv8, một mô hình mạng nơ-ron sâu (deep neural network) được thiết kế để giải quyết bài toán nhận diện đối tượng một cách chính xác và hiệu quả trong hệ thống bãi đỗ xe thông minh.



Hình 5.1: Hình ảnh kiến trúc YOLOv8

a. Cấu trúc mô hình YOLOv8

Mô hình YOLOv8 bao gồm ba phần chính: *Backbone*, *Neck* và *Head*. Mỗi phần đảm nhiệm một vai trò riêng trong việc xử lý và dự đoán đối tượng.

Backbone

Backbone chịu trách nhiệm trích xuất đặc trưng từ ảnh đầu vào với kích thước $640 \times 640 \times 3$. Giai đoạn đầu tiên (stem) bao gồm hai lớp convolution có kernel size 3 và stride 2 nhằm giảm kích thước không gian và trích xuất đặc trưng cơ bản. Các khối C2f trong backbone (blocks số 2, 4, 6, 8) sử dụng *shortcut connection* để truyền thông tin hiệu quả. Việc giảm kích thước được thực hiện bằng các lớp convolution với stride 2.

Neck

Neck kết hợp các đặc trưng từ nhiều tầng khác nhau để tăng khả năng nhận diện đối tượng ở nhiều kích thước. Thành phần nổi bật là khối SPPF (Spatial Pyramid Pooling Fast), giúp thu nhận thông tin ở nhiều tỉ lệ khác nhau. Neck cũng sử dụng các kỹ thuật *upsample* (tăng kích thước đặc trưng) và *concat* (kết hợp đặc trưng)

để truyền tải thông tin. Các khối C2f ở phần này (blocks 12, 15, 18, 21) không sử dụng shortcut như ở backbone.

Head

Phần Head là nơi thực hiện dự đoán cuối cùng. Có ba nhánh đầu ra tương ứng với ba kích thước đặc trưng: 80×80 , 40×40 và 20×20 (P3, P4, P5). Mỗi nhánh sử dụng một lớp Detect để xác định vị trí và lớp của các đối tượng.

Các biến thể mô hình

YOLOv8 có nhiều biến thể tùy theo độ phức tạp và hiệu suất, được xác định bởi các tham số `depth_multiple` (d), `width_multiple` (w), và `max_channels` (mc).

Biến thể	d	w	mc
n	0.33	0.25	1024
s	0.33	0.50	1024
m	0.67	0.75	768
l	1.00	1.00	512
x	1.00	1.25	512

Tổng kết

YOLOv8 sử dụng khối C2f thay thế cho CSP truyền thống, với shortcut chỉ áp dụng ở backbone. Neck kết hợp thông tin đa tầng qua concat và upsampling, trong khi Head dự đoán ở ba tỉ lệ giúp mô hình nhận diện tốt cả đối tượng nhỏ và lớn, đặc biệt là biển số xe, trong môi trường thực tế với độ chính xác và tốc độ cao.

b. Huấn luyện các mô hình YOLOv8, sử dụng dữ liệu từ Roboflow

```
[ ] echo 'train: /content/drive/MyDrive/Colab/Volov8/LicensePlate/train/images' >> LicensePlate/plate.yaml
echo 'val: /content/drive/MyDrive/Colab/Volov8/LicensePlate/test/images' >> LicensePlate/plate.yaml
echo 'nc: 1' >> LicensePlate/plate.yaml
echo 'name: ['LicensePlate']' >> LicensePlate/plate.yaml

lyolo detect train data=LicensePlate/plate.yaml model=last.pt epochs=20 imgsz=640

Ultralytics 8.3.4 Python-3.10.12 torch-2.4.1+cu121 CPU (Intel Xeon 2.20GHz)
engine/trainer: task=detect, mode=train, model=yolov8n.pt, data=LicensePlate/plate.yaml, epochs=20, time=None, patience=100, batch=16, imgsz=640, save=True, save_period=-1, cache=False, device=None,
Downloading https://ultralytics.com/assets/Arial.ttf to /root/.config/Ultralytics/Arial.ttf...
100% 755k/755k [00:00<00:00, 14.9MB/s]
Overriding model.yaml nc=80 with nc=1

from n  params  module  arguments
0      1      464    ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1      1      4672   ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2      1      7360   ultralytics.nn.modules.block.C2f  [32, 32, 1, True]
3      1      18560  ultralytics.nn.modules.conv.Conv  [32, 64, 3, 2]
4      1      49664  ultralytics.nn.modules.block.C2f  [64, 64, 2, True]
5      1      73984  ultralytics.nn.modules.conv.Conv  [64, 128, 3, 2]
6      1      197632 ultralytics.nn.modules.block.C2f  [128, 128, 2, True]
7      1      295424 ultralytics.nn.modules.conv.Conv  [128, 256, 3, 2]
8      1      460288 ultralytics.nn.modules.block.C2f  [256, 256, 1, True]
9      1      164608 ultralytics.nn.modules.block.SPPF  [256, 256, 5]
10     1      0      torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
11     [-1, 6] 1      0      ultralytics.nn.modules.conv.Concat  [1]
12     1      148224  ultralytics.nn.modules.block.C2f  [384, 128, 1]
13     1      0      torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
14     [-1, 4] 1      0      ultralytics.nn.modules.conv.Concat  [1]
15     1      37248  ultralytics.nn.modules.block.C2f  [192, 64, 1]
16     1      36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
17     [-1, 12] 1      0      ultralytics.nn.modules.conv.Concat  [1]
18     1      123648 ultralytics.nn.modules.block.C2f  [192, 128, 1]
19     1      147712 ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
20     [-1, 9] 1      0      ultralytics.nn.modules.conv.Concat  [1]
21     1      493056 ultralytics.nn.modules.block.C2f  [384, 256, 1]
22     [15, 18, 21] 1 4308672 ultralytics.nn.modules.head.Detect  [1, [64, 128, 256]]

Model summary: 249 layers, 2,690,403 parameters, 2,690,387 gradients, 6.9 GFLOPS
```

Hình 5.2: Huấn luyện mô hình nhận diện biển xe

Tập dữ liệu gồm 10125 hình ảnh biển số xe được thu thập đa dạng và chia vào các folder chính train, test. Đường link truy cập <https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e>

```
[ ] echo 'train: /content/drive/MyDrive/Colab/Yolov8/OcrData/train/images' >> OcrData/ocrdata.yaml
[ ] echo 'val: /content/drive/MyDrive/Colab/Yolov8/OcrData/val/images' >> OcrData/ocrdata.yaml
[ ] echo 'nc: 31' >> OcrData/ocrdata.yaml
[ ] echo "names: ['0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F','G','H','I','J','K','L','M','N','P','R','S','T','U','V','X','Y','Z']" >> OcrData/ocrdata.yaml

[ ] !yolo detect train data=OcrData/ocrdata.yaml model=last.pt epochs=5 imgsz=640

[ ] Ultralytics YOLOv8.0.199 Python-3.10.12 torch-2.0.1+cu118 CPU (Intel Xeon 2.20GHz)
[ ] engine/trainer: task=detect, mode=train, model=yolov8n.pt, data=OcrData/ocrdata.yaml, epochs=5, patience=50, batch=16, imgsz=640, save=True, save_period=-1, cache=False, device=None, workers=8
[ ] Downloading https://ultralytics.com/assets/Arial.ttf to /root/.config/Ultralytics/Arial.ttf...
[ ] 100% 755K/755K [00:00<00:00, 13.2MB/s]
[ ] Overriding model.yaml nc=80 with nc=31

from n      params module                                arguments
0         -1 1      464  ultralytics.nn.modules.conv.Conv                [3, 16, 3, 2]
1         -1 1     4672 ultralytics.nn.modules.conv.Conv                [16, 32, 3, 2]
2         -1 1     7360 ultralytics.nn.modules.block.C2f                [32, 32, 1, True]
3         -1 1    18560 ultralytics.nn.modules.conv.Conv                [32, 64, 3, 2]
4         -1 2    49664 ultralytics.nn.modules.block.C2f                [64, 64, 2, True]
5         -1 1    73984 ultralytics.nn.modules.conv.Conv                [64, 128, 3, 2]
6         -1 2   197632 ultralytics.nn.modules.block.C2f                [128, 128, 2, True]
7         -1 1   295424 ultralytics.nn.modules.conv.Conv                [128, 256, 3, 2]
8         -1 1   460288 ultralytics.nn.modules.block.C2f                [256, 256, 1, True]
9         -1 1   164608 ultralytics.nn.modules.block.SPPF                [256, 256, 5]
10        -1 1         0 torch.nn.modules.upsampling.Upsample          [None, 2, 'nearest']
11        [-1, 6] 1         0 ultralytics.nn.modules.conv.Conv                  [1]
12        -1 1   148224 ultralytics.nn.modules.block.C2f                [384, 128, 1]
13        -1 1         0 torch.nn.modules.upsampling.Upsample          [None, 2, 'nearest']
14        [-1, 4] 1         0 ultralytics.nn.modules.conv.Conv                  [1]
15        -1 1    37248 ultralytics.nn.modules.block.C2f                [192, 64, 1]
16        -1 1    36992 ultralytics.nn.modules.conv.Conv                [64, 64, 3, 2]
17        [-1, 12] 1         0 ultralytics.nn.modules.conv.Conv                  [1]
18        -1 1   123648 ultralytics.nn.modules.block.C2f                [192, 128, 1]
19        -1 1   147712 ultralytics.nn.modules.conv.Conv                [128, 128, 3, 2]
20        [-1, 9] 1         0 ultralytics.nn.modules.conv.Conv                  [1]
21        -1 1    49366 ultralytics.nn.modules.block.C2f                [384, 256, 1]
22        [15, 18, 21] 1 757357 ultralytics.nn.modules.head.Detect              [31, [64, 128, 256]]

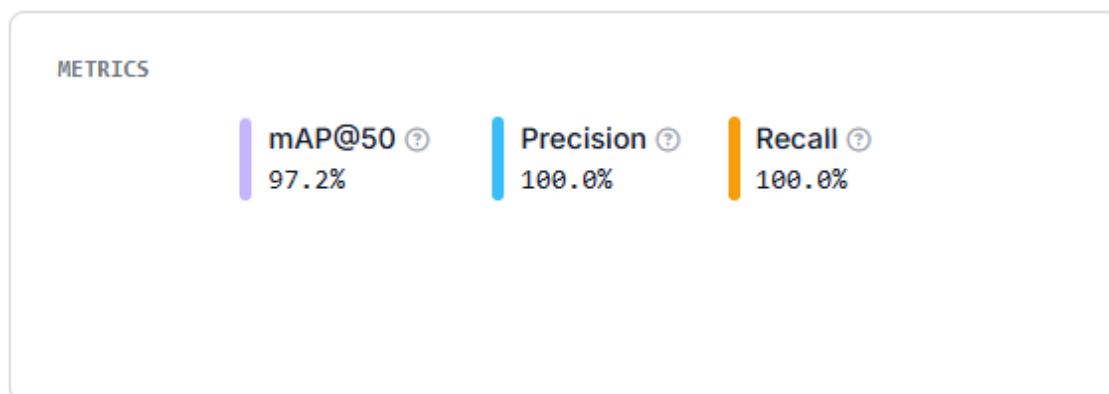
Model summary: 225 layers, 3016893 parameters, 3016877 gradients, 8.2 GFLOPs
```

Hình 5.3: Huấn luyện mô hình nhận diện ký tự trong biển số xe

Tập dữ liệu gồm 500 ảnh chứa 31 ký tự số (0–9) và chữ cái (A–Z) thường xuất hiện trên biển số xe ở Việt Nam. Đường link truy cập <https://universe.roboflow.com/hust-ac6st/genplate>

5.1.3 Kết quả đạt được

Việc áp dụng giải pháp này sẽ giúp tự động hóa quá trình nhận diện biển số xe, mang lại hiệu quả cao trong việc xử lý dữ liệu hình ảnh, tiết kiệm thời gian và giảm thiểu sai sót so với phương pháp thủ công. Các hệ thống nhận diện biển số có thể đạt được độ chính xác cao.



Hình 5.4: Đánh giá độ chính xác

5.2 Kết nối phần cứng và phần mềm

5.2.1 Giới thiệu vấn đề

Trong các hệ thống AIoT như nhận diện biển số xe và quản lý bãi đỗ thông minh, việc kết nối giữa phần cứng và phần mềm đóng vai trò then chốt. Quá trình này đòi hỏi sự đồng bộ và tích hợp chặt chẽ giữa các thiết bị phần cứng (như camera, cảm biến siêu âm, module RFID) với phần mềm xử lý (bao gồm thuật toán nhận diện, giao diện người dùng và hệ thống điều khiển). Mục tiêu là đảm bảo hệ thống vận hành ổn định, phản hồi thời gian thực và mang lại trải nghiệm tối ưu cho người dùng.

5.2.2 Giải pháp triển khai

Hệ thống được xây dựng với kiến trúc tích hợp đa thành phần, cụ thể như sau:

- **Cảm biến và giao tiếp phần cứng:** Các cảm biến siêu âm được kết nối với vi điều khiển ESP32 để phát hiện trạng thái các vị trí đỗ xe (có xe / không xe). Đồng thời, camera ESP32-CAM được sử dụng để ghi nhận hình ảnh xe tại thời điểm vào/ra, truyền dữ liệu hình ảnh về server xử lý qua giao thức HTTP.

```
void handleJPG(void)
{
    WiFiClient client = server.client();

    if (!client.connected()) return;
    camera_fb_t* fb = esp_camera_fb_get();
    client.write(JHEADER, jhdLen);
    client.write((char*)fb->buf, fb->len);
    esp_camera_fb_return(fb);
}
```

Hình 5.5: Hàm gửi 1 khung hình dạng JPEG qua HTTP

- **Xử lý dữ liệu và thuật toán:** Server nhận hình ảnh từ camera và thực hiện xử lý bằng mô hình học sâu YOLOv8 để phát hiện và nhận diện biển số xe. Quá trình xử lý sử dụng thư viện OpenCV và mô hình đã huấn luyện từ Roboflow. Kết quả được lưu trữ và đồng bộ lên hệ thống quản lý trung tâm.

```
# Load model YOLO 1 lần duy nhất khi import
plate_detector = YOLO("model/LP_detector.pt")
char_detector = YOLO("model/LP_ocr.pt")

✓ def run_plate_recognition(image_bytes):
    """
    Nhận diện biển số từ ảnh (dạng bytes)
    Trả về: dict {"plate": "61A12345"}
    """

    # Giải mã ảnh từ bytes về OpenCV format
    image_np = np.frombuffer(image_bytes, np.uint8)
    frame = cv2.imdecode(image_np, cv2.IMREAD_COLOR)

    if frame is None:
        raise ValueError("Không thể giải mã ảnh từ bytes.")

    # Gọi hàm xử lý ảnh đã có sẵn
    _, plate_text = process_frame(frame, plate_detector, char_detector)

    return {
        "plate": plate_text
    }
```

Hình 5.6: Hàm xử lý nhận diện biển số xe

- **Giao diện người dùng:** Giao diện web được thiết kế trực quan, cho phép hiển thị trạng thái các vị trí đỗ xe theo thời gian thực, biển số xe đã nhận diện, và thời gian xe vào/ra. Người quản lý có thể theo dõi, tìm kiếm hoặc điều phối xe từ giao diện này.
- **Tích hợp điều khiển và phản hồi:** Khi hệ thống nhận diện được biển số hợp lệ, tín hiệu điều khiển sẽ được gửi từ phần mềm đến vi điều khiển (ESP32 hoặc Arduino) để mở cổng tự động qua relay. Việc này giúp đảm bảo quy trình ra/vào diễn ra hoàn toàn tự động, nhanh chóng và chính xác.

```

void buttonGateInHandler(AceButton* b, uint8_t e, uint8_t s) {
    if (e == AceButton::kEventPressed) {
        delay(1000);
        Serial.println("[IN] Nhan cam bien vao...");
        if (sendGateEvent("entry", "sensor")) {
            servo1.write(0);
        }
    } else if (e == AceButton::kEventReleased) {
        Serial.println("[IN] Nhan.");

        delay(2000);
        servo1.write(90);
    }
}

void buttonGateOutHandler(AceButton* b, uint8_t e, uint8_t s) {
    if (e == AceButton::kEventPressed) {
        // Serial.println("[OUT] Nhan cam bien ra...");
        if (sendGateEvent("exit", "sensor")) {
            servo2.write(90);
            check = 1;
        }
    } else if ((e == AceButton::kEventReleased) && (check == 1)) {
        delay(2000);
        check = 0;
        servo2.write(0);
    }
}

```

Hình 5.7: ESP nhận lệnh đóng mở cổng

```

if mode == "sensor":
    try:
        profile = Profile.objects.get(license_plate__iexact=plate_text)
    except Profile.DoesNotExist:
        if position == "exit":
            guest_log = AccessHistory.objects.filter(
                license_plate__iexact=plate_text,
                check_out__isnull=True
            ).first()
            broadcast_access_event({"status": "chua_co_thong_tin", "position": position, "license_plate": plate_text, "rfid": rfid_id, "check_in": now.isoformat(), "check_out": now.isoformat()})
            return JsonResponse({"trang_thai": "that_bai", "thong_diep": "khong_tim_thay_nguoi_dung", "bien_so": plate_text, "image_url": image_url}, status=404)

        user = profile.user
        active_log = AccessHistory.objects.filter(user=user, check_out__isnull=True).first()

        if position == "entry":
            if active_log:
                broadcast_access_event({"status": "xe_da_trong_bai", "position": position, "license_plate": plate_text, "rfid": rfid_id, "check_in": active_log.check_in.isoformat(), "check_out": now.isoformat()})
                return JsonResponse({"trang_thai": "that_bai", "thong_diep": "xe_da_trong_bai", "bien_so": plate_text, "image_url": image_url}, status=400)
            AccessHistory.objects.create(user=user, license_plate=plate_text, rfid_code=rfid_id, check_in=now)
            broadcast_access_event({"status": "checkin_thanh_cong", "position": position, "license_plate": plate_text, "rfid": rfid_id, "check_in": now.isoformat(), "check_out": now.isoformat()})
            return JsonResponse({"trang_thai": "thanh_cong", "thong_diep": "checkin_thanh_cong", "bien_so": plate_text, "image_url": image_url})

        elif position == "exit":
            if not active_log:
                broadcast_access_event({"status": "xe_chua_vao", "position": position, "license_plate": plate_text, "rfid": rfid_id, "image_url": image_url})
                return JsonResponse({"trang_thai": "that_bai", "thong_diep": "xe_chua_vao", "bien_so": plate_text, "image_url": image_url}, status=400)

            minutes = int((now - active_log.check_in).total_seconds() // 60)
            total_fee = minutes * getattr(settings, "COST", 1000)

            if profile.balance < total_fee:
                broadcast_access_event({"status": "so_du_khong_du", "position": position, "license_plate": plate_text, "rfid": rfid_id, "check_in": active_log.check_in.isoformat(), "check_out": now.isoformat()})
                return JsonResponse({"trang_thai": "that_bai", "thong_diep": "so_du_khong_du", "bien_so": plate_text, "so_du": float(profile.balance), "so_tien_can": total_fee, "image_url": image_url})

            profile.balance -= total_fee
            profile.save()
            active_log.check_out = now
            active_log.fee = total_fee

```

Hình 5.8: Server gửi thông điệp đóng mở cổng

5.2.3 Kết quả đạt được

Việc tích hợp đồng bộ phần cứng và phần mềm trong hệ thống giúp đảm bảo xử lý hiệu quả và phản hồi tức thời với các tình huống thực tế. Hệ thống hoạt động ổn định, hỗ trợ tự động nhận diện và mở cổng khi biển số hợp lệ được phát hiện. Nhờ đó, quy trình kiểm soát bãi đỗ xe được tối ưu hóa, giảm thiểu sai sót, tiết kiệm nhân lực và tăng độ chính xác. Đây là bước tiến quan trọng trong việc ứng dụng AIoT vào giải pháp giao thông thông minh tại các khu dân cư, trường học và trung tâm thương mại.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã xây dựng thành công mô hình hệ thống quản lý bãi đỗ xe thông minh ứng dụng công nghệ AIoT, kết hợp giữa phần cứng nhúng (ESP32, cảm biến siêu âm, camera ESP32-CAM, RFID) và phần mềm xử lý (YOLOv8, OpenCV, giao diện Web). Hệ thống hỗ trợ nhận diện biển số xe, cập nhật trạng thái bãi đỗ theo thời gian thực và điều khiển tự động mở cổng khi xe ra vào.

So với các giải pháp hiện có trên thị trường, sản phẩm có ưu điểm về chi phí thấp, dễ triển khai và phù hợp với quy mô bãi đỗ xe vừa và nhỏ tại Việt Nam. Mô hình có khả năng mở rộng, dễ tích hợp với nhận diện khuôn mặt trong tương lai.

Trong quá trình thực hiện đồ án, tôi đã nắm được kiến thức liên quan đến lập trình vi điều khiển, xử lý ảnh, mô hình học sâu (YOLO), đồng thời phát triển kỹ năng làm việc với hệ thống nhúng và thiết kế giao diện người dùng. Tuy nhiên, một số điểm còn hạn chế như chưa tối ưu tốc độ xử lý thời gian thực, độ chính xác có thể bị ảnh hưởng trong điều kiện ánh sáng yếu hoặc biển số mờ.

Những bài học quan trọng rút ra bao gồm: khả năng tổ chức công việc theo giai đoạn, làm việc với phần cứng thực tế, kết hợp kiến thức liên ngành (AI, IoT, web server, nhúng), và kiên nhẫn xử lý lỗi phát sinh khi tích hợp hệ thống.

6.2 Hướng phát triển

Để hoàn thiện và nâng cấp sản phẩm, một số hướng phát triển khả thi trong tương lai bao gồm:

- Tối ưu thuật toán nhận diện để cải thiện độ chính xác và tốc độ xử lý trong điều kiện thực tế (ánh sáng yếu, biển số mờ, nhiều xe đồng thời).
- Mở rộng mô hình để quản lý nhiều cổng ra vào, nhiều bãi xe cùng lúc thông qua nền tảng web tập trung (cloud-based).
- Xây dựng thêm ứng dụng di động để người dùng đặt chỗ, xem tình trạng bãi đỗ theo thời gian thực.
- Nâng cấp phần cứng sử dụng camera chất lượng cao hơn, bổ sung đèn hồng ngoại giúp nhận diện tốt vào ban đêm.

Những định hướng trên sẽ giúp hệ thống trở nên hoàn thiện, thông minh và phù hợp hơn với các ứng dụng thực tế tại khu dân cư, trường học, bệnh viện hoặc trung tâm thương mại trong bối cảnh đô thị hóa ngày càng phát triển.

TÀI LIỆU THAM KHẢO

[1] *YOLOv8 Architecture*. [Online]. Available: <https://arxiv.org/abs/2501.13400> (visited on 12/04/2025).

[2] *Django Web Framework*. [Online]. Available: <https://www.djangoproject.com/> (visited on 12/04/2025).

[3] *SQLite3 Database*. [Online]. Available: <https://sqlite.org/> (visited on 12/04/2025).

[4] *RC522 RFID Module*. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf> (visited on 12/04/2025).

[5] *ESP32-WROOM-32*. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf (visited on 12/04/2025).

[6] *Model MVC Django*. [Online]. Available: <https://iq.opengenus.org/model-view-controller-django/> (visited on 12/04/2025).

[7] *Bootstrap documentation*. [Online]. Available: <https://getbootstrap.com/docs/5.3/getting-started> (visited on 12/04/2025).